

Macchine di Turing

- Macchine di Turing
- Linguaggi Ricorsivi e Ricorsivamente Enumerabili

(basato su lucidi di Jeffrey Ullman disponibili su
<http://infolab.stanford.edu/~ullman/ialc.html>)

Definizione di "Calcolabile": La Tesi dice che la nozione intuitiva di "calcolabilità" (risolvibilità tramite una procedura algoritmica, passo-passo) coincide esattamente con la nozione formale di ciò che può essere calcolato da una Macchina di Turing (MT).

È cruciale notare che si tratta di una Tesi e non di un Teorema. Non è dimostrabile formalmente, poiché mette in relazione un concetto informale ("intuitivamente calcolabile") con un concetto formale (risolvibile da una MT). È accettata universalmente per la schiacciante evidenza che ogni altro modello di calcolo finora proposto ha dimostrato di avere esattamente la stessa potenza computazionale della MT.

Tesi di Turing-Church

- Turing studiò i meccanismi basilari del calcolo automatico, elaborò un suo modello di calcolo (le Macchine di Turing) ed espresse il proprio pensiero nel seguente modo:
 - **"SE UN PROBLEMA E' INTUITIVAMENTE CALCOLABILE (cioè risolvibile eseguendo una procedura) ALLORA ESISTE UNA TURING MACHINE CHE LO RISOLVE"**
- Vediamo cosa sono queste Turing Machine...

Algoritmo: Un insieme finito e non ambiguo di istruzioni per risolvere un problema. Deve terminare (fermarsi) su ogni input possibile, fornendo sempre una risposta (Sì o NO) o un risultato. Corrisponde a una Macchina di Turing che si ferma sempre.
Procedura: Un insieme finito e non ambiguo di istruzioni per risolvere un problema. Non è garantito che termini su tutti gli input. Si ferma se l'input è nel linguaggio/risolve il problema, ma può non fermarsi (andare in loop) se l'input non è nel linguaggio/non risolve il problema. Corrisponde a una Macchina di Turing che non è garantito che si fermi sempre.

Macchina di Turing: assomiglia ad un automa a pila, ma si usa un nastro (simile ad uno stack, ma posso accedere a qualsiasi posizione, come fosse una memoria). Il nastro è illimitato. Per semplificare, l'input se lo trova sul nastro (per questo Sigma è sottoinsieme di Gamma)

La macchina di Turing è deterministica: determinismo non è limitante, perché posso simulare tanti nastri con un solo nastro, essendo infinito: Il nastro infinito permette alla DTM di memorizzare tutte le possibili scelte e simulare efficacemente la parallelizzazione della NTM. (La Macchina di Turing è definita come deterministica nella sua forma standard, e il non determinismo non ne aumenta l'espressività)

Definizione formale di Macchina di Turing

Una TM è una 7-pla $(Q, \Sigma, \Gamma, \delta, q_0, B, F)$ con:

- Un insieme **finito** di **stati** Q L'insieme di tutti gli stati interni in cui si può trovare la MT. Simile agli stati di un Automa a Stati Finiti.

- Un **alfabeto di input** Σ L'insieme finito di simboli che possono comparire nella stringa di input iniziale. Sigma non include il simbolo blank (B).

- Un **alfabeto del nastro** Γ (contiene Σ) L'insieme finito di simboli che possono essere letti e scritti sul nastro. Cruciale: Sigma sottoinsieme di Gamma. Gamma deve contenere Sigma e il simbolo blank B. Dunque: B appartiene a Gamma e Sigma sottoinsieme di Gamma.

- Una funzione di **transizione** δ

- Uno **stato iniziale** $q_0 \in Q$ Lo stato in cui si trova la MT all'inizio del calcolo. q_0 appartiene a Q .

- Un simbolo **blank** $B \in \Gamma - \Sigma$ Un simbolo speciale che denota una cella vuota del nastro. B non appartiene a Sigma. L'intero nastro, tranne la porzione iniziale contenente l'input, è riempito con B.

- Tutto il nastro, meno l'input, all'inizio è blank

- Un insieme di **stati finali** $F \subseteq Q$

L'insieme degli stati in cui la MT entra per indicare che la stringa di input è stata accettata. F sottoinsieme di Q. Se la MT si ferma in uno stato q appartenente a F, la computazione ha successo.

La regola che governa il movimento della macchina. Questo è il cuore della MT: dato uno stato corrente q appartenente a Q e un simbolo letto X appartenente a Γ dal nastro, la MT esegue tre azioni in sequenza:

- Cambia Stato: Passa a un nuovo stato q' appartenente a Q .
- Scrive: Sostituisce il simbolo letto X con un nuovo simbolo Y appartenente a Γ .
- Muove: Sposta la testina di lettura/scrittura a sinistra (L) o a destra (R).

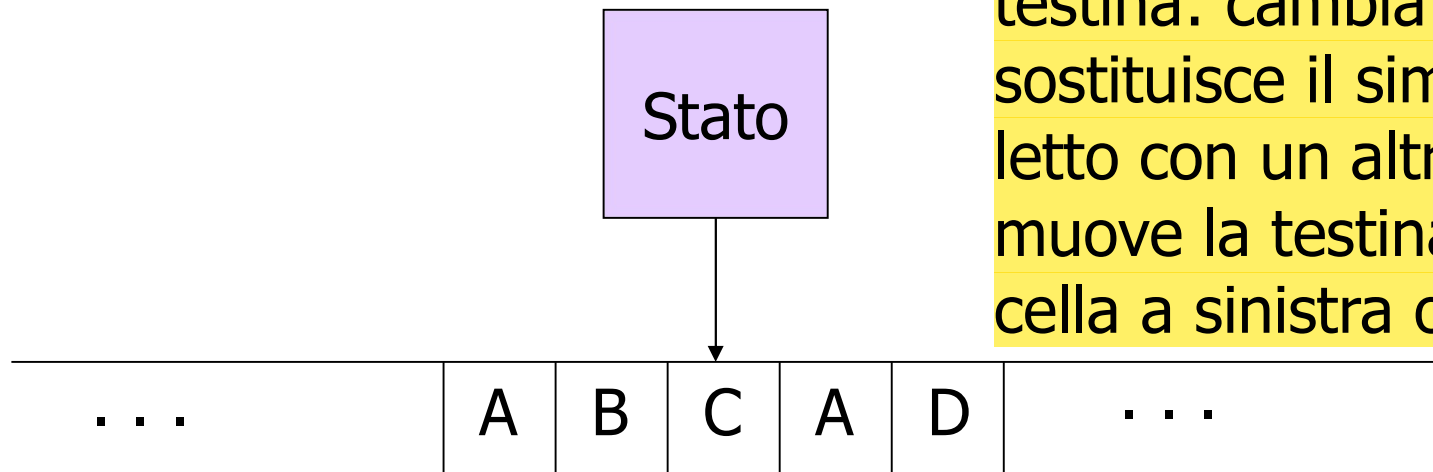
Questa combinazione di memoria illimitata (nastro infinito) e logica di controllo finita (funzione delta e stati Q) è ciò che rende la Macchina di Turing il modello di calcolo più potente. È l'astrazione perfetta di un computer moderno (memoria di massa illimitata) che esegue un programma (logica di controllo finita).

Macchine di Turing

Questo descrive la funzione di transizione delta che governa il comportamento della macchina. L'azione è deterministica nella MT standard (data una coppia (Stato, Simbolo letto), c'è una sola azione possibile). L'azione si basa su due input e produce tre output (Nuovo Stato, Nuovo Simbolo Scritto, Direzione del Movimento)

Azione:

funzione dello **stato** e del **simbolo** sotto la testina: cambia stato, sostituisce il simbolo letto con un altro, e muove la testina di una cella a sinistra o a destra



Sebbene la MT abbia un'unità di controllo finita (un numero finito di stati Q), la sua memoria (il nastro) è potenzialmente infinita in una direzione o in entrambe. È questa risorsa illimitata che le permette di eseguire calcoli che richiedono una quantità di memoria arbitrariamente grande.

Alfabeto Finito (Gamma): Nonostante la memoria sia infinita, il tipo di informazione che può essere memorizzata in ogni cella è limitato ai simboli dell'Alfabeto del Nastro (Gamma). Come abbiamo visto, Gamma include l'alfabeto di input (Sigma) e il simbolo blank (B).

Nastro infinito con celle che contengono simboli appartenenti ad un **alfabeto finito** Γ

Convenzioni

- $a, b, \dots$ sono simboli in input
- $\dots, X, Y, Z$ sono simboli del nastro
- $\dots, w, x, y, z$ sono stringhe di simboli in input
- $\alpha, \beta, \dots$ sono stringhe di simboli del nastro

Una MT si ferma (o blocca) se e solo se:
- Entra in uno stato di accettazione/finale (q appartenente ad F): In molte definizioni standard, non esistono transizioni definite per gli stati di accettazione. Il calcolo si ferma e la stringa è ACCETTATA.
- Raggiunge uno stato non-finale e un simbolo (Z) per i quali la delta non è definita: La macchina non sa cosa fare, si blocca e la stringa è RIFIUTATA.

La funzione di transizione

- Dati due argomenti:

- Uno stato q in Q
- Un simbolo Z del nastro appartenente a Γ

delta è definita come
Funzione Parziale
perché può bloccarsi

■ $\delta(q, Z)$ è indefinito oppure è **una** tripla dalla forma (p, Y, D) .

MT Deterministica: Poiché stiamo parlando di una MT deterministica, per ogni coppia (q, Z) , c'è al massimo una possibile transizione.
Terminazione: Il fatto che $\delta(q, Z)$ possa essere indefinita è il meccanismo formale con cui la Macchina di Turing si ferma.

- p è uno stato
- Y è un nuovo simbolo del nastro
- D è una **direzione**, L o R

di spostamento della testina

Azioni della TM

ha un'unica mossa, non un insieme (essendo deterministico)

- Se $\delta(q, Z) = (p, Y, D)$ allora, la TM nello stato q , che legge Z sul nastro:
 - Cambia lo stato in p
 - Rimpiazza Z con Y sul nastro (può riscrivere lo stesso simbolo, anziché aggiungere un nuovo simbolo sul nastro)
 - Muove la testina di una cella lungo la direzione D
 - Se D è L muove a sinistra
 - Se D è R muove a destra

Esempio: Macchina di Turing

- Si consideri una TM che legge la stringa in input in attesa di trovare un 1
 - la testina si trova inizialmente sul primo carattere dell'input
- Se ne trova uno, lo cambia in 0, entra in uno stato finale f , e si ferma
- Se raggiunge un blank, lo cambia in 1, e si sposta a sinistra

All'inizio, in TM, viene messa tutta la stringa in input sul nastro e la testina si posiziona sul primo carattere della stringa di input sul nastro

Accetta la stringa

(per accettare, basta che si trovi su uno stato di accettazione: non per forza deve consumare tutto l'input, che è già presente sul nastro)

Se la stringa non contiene un 1, arrivato alla fine della stringa dove ho carattere Blank, la TM crea un 1 (quindi le stringhe che la TM accetta sono infinite)

Esempio: Macchina di Turing (2)

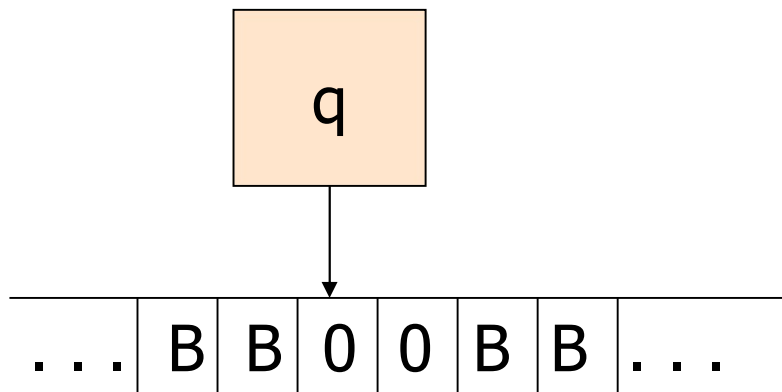
- Stati = {q (stato iniziale), f (finale)}
- Simboli in input = {0, 1}
- Simboli del nastro = {0, 1, B}
- $\delta(q, 0) = (q, 0, R)$ Se trova lo 0, rimane sullo stesso stato, rimette lo 0 dove si trova con la testina e va avanti con la testina
- $\delta(q, 1) = (f, 0, R)$ Se trova l'1, va in stato di accettazione, mette lo 0 dove si trova con la testina e va avanti (ACCETTA la stringa)
- $\delta(q, B) = (q, 1, L)$ Se trova il carattere Blank, rimane in stato q, sostituisce il Blank con 1 e torna indietro con la testina

Simulazione di TM

$$\delta(q, 0) = (q, 0, R)$$

$$\delta(q, 1) = (f, 0, R)$$

$$\delta(q, B) = (q, 1, L)$$

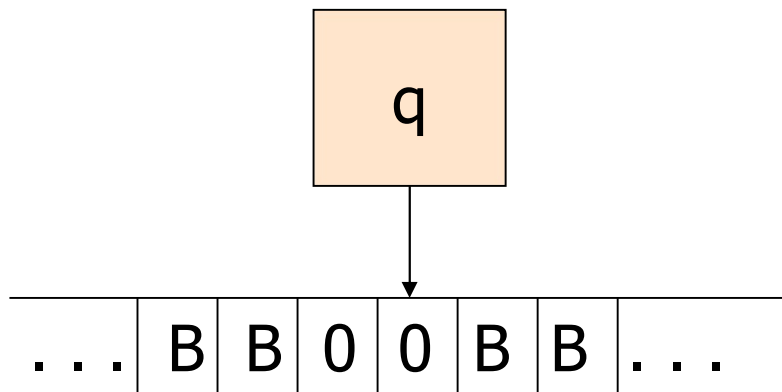


Simulazione di TM

$$\delta(q, 0) = (q, 0, R)$$

$$\delta(q, 1) = (f, 0, R)$$

$$\delta(q, B) = (q, 1, L)$$

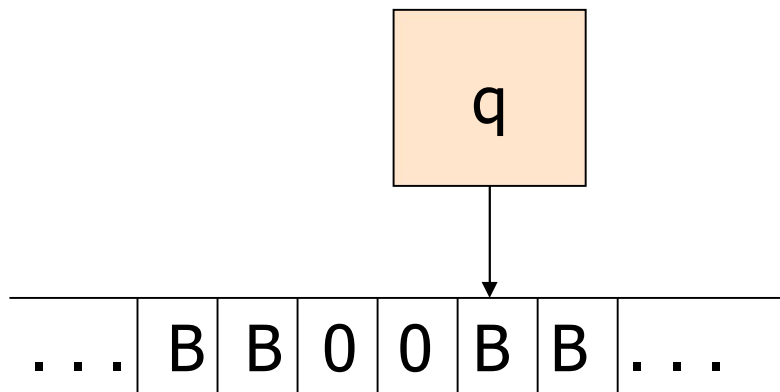


Simulazione di TM

$$\delta(q, 0) = (q, 0, R)$$

$$\delta(q, 1) = (f, 0, R)$$

$$\delta(q, B) = (q, 1, L)$$

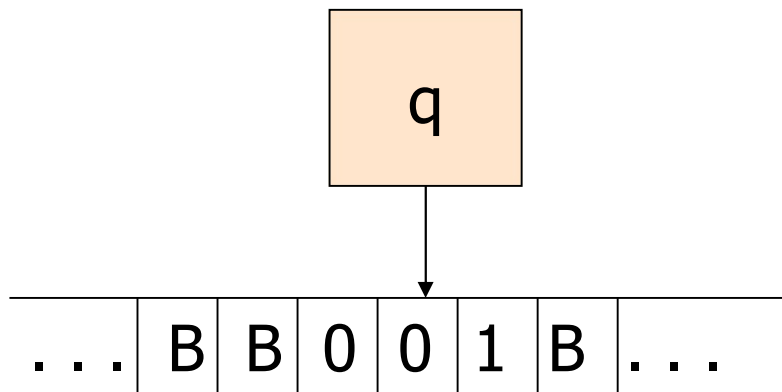


Simulazione di TM

$$\delta(q, 0) = (q, 0, R)$$

$$\delta(q, 1) = (f, 0, R)$$

$$\delta(q, B) = (q, 1, L)$$

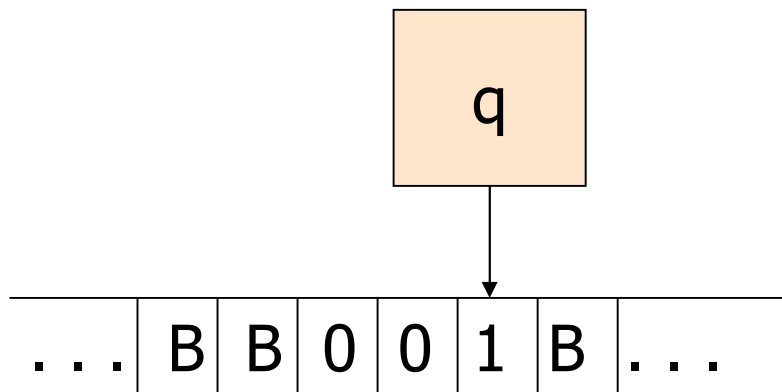


Simulazione di TM

$$\delta(q, 0) = (q, 0, R)$$

$$\delta(q, 1) = (f, 0, R)$$

$$\delta(q, B) = (q, 1, L)$$

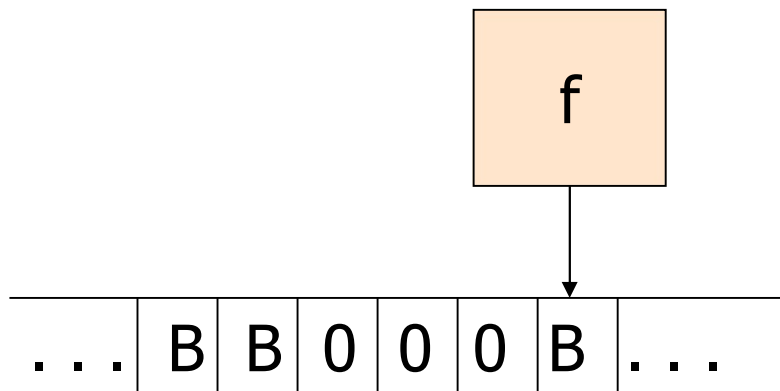


Simulazione di TM

$$\delta(q, 0) = (q, 0, R)$$

$$\delta(q, 1) = (f, 0, R)$$

$$\delta(q, B) = (q, 1, L)$$



La TM è in uno stato
finale: si ferma e
accetta

Descrizione Istantanea (ID) di una Turing Machine

- Inizialmente, una TM ha un nastro che consiste di una **stringa di simboli di input**, circondata attornata da una **infinità di blank** in entrambe le direzioni
- La TM è nello **stato iniziale**, e la testina è sul **simbolo in input più a sinistra**

ID di una TM – (2)

- In generale, una ID è rappresentata tramite una **stringa** $\alpha q \beta$, dove:
 - $\alpha \beta$ è il contenuto del nastro
 - a sinistra di $\alpha \beta$ e a destra di $\alpha \beta$ si assume che il nastro contenga tutte B
 - q è lo stato della TM
 - la testina è posizionata sul primo carattere di β
 - se β è vuota sotto la testina c'è B

Rappresenta la porzione non-blank (significativa) del nastro. Tutto ciò che è al di fuori di alpha beta è considerato il simbolo blank (B).

La ID è $\alpha q \epsilon$ (o semplicemente αq). In questo caso, si assume che la cella sotto la testina, e tutte le celle a destra, contengano il simbolo blank (B). Quindi, la MT agisce come se stesse leggendo B. Formalmente, l'ID αq è equivalente a $\alpha q B$.

ID di una TM– (3)

- Come per i PDA usiamo i simboli $\vdash$ e $\vdash^*$ per rappresentare i **passaggi di ID**: rispettivamente, “diventa in una mossa” e “diventa in zero o più mosse”
- Esempio: le mosse dell’ esempio precedente sono:
 $q00\vdash 0q0\vdash 00q\vdash 0q01\vdash 00q1\vdash 000f$


Configurazione iniziale


Configurazione finale

Definizione Formale di Mossa

alfa e beta rimangono invariati

La TM cambia stato e carattere sul nastro, spostandosi a destra

Caso in cui la testina si sposta a destra

1. Se $\delta(q, Z) = (p, Y, R)$, allora

- $\alpha q Z \beta \vdash \alpha Y p \beta$

(allora anche beta è costituito da blank)

- se Z è B, allora anche $\alpha q \vdash \alpha Y p$

La TM cambia stato e carattere sul nastro, spostandosi a sinistra

Caso in cui la testina si sposta a sinistra

2. Se $\delta(q, Z) = (p, Y, L)$, allora

- $\alpha q Z \beta \vdash \gamma p X Y \beta$ con $\alpha = \gamma X$, oppure $\alpha = \gamma = \varepsilon$ e $X = B$

X è l'ultimo carattere di alfa e gamma sono i primi caratteri di alfa

- se Z è B, allora anche $\alpha q \vdash \gamma p X Y$ con stessa cond.

Quindi o alfa ha caratteri nella stringa e quindi la testina si trova dentro la stringa di input oppure la testina si trova all'inizio della stringa di input e quindi alfa vuota

Se alfa è stringa vuota, X è un carattere Blank e gamma è epsilon

Linguaggio di una TM

- Una TM M definisce un linguaggio per stato finale, come al solito:

← ■ $L(M) = \{w \mid q_0 w \vdash^* \alpha q \beta, \text{ con } q \text{ finale}\}$

Non importa dove si trova la testina e cosa è alfa e beta (mi basta che sia in stato finale per accettare la stringa)

Quelle stringhe che da una configurazione iniziale mi porta in 0 o più passaggi di configurazione in qualsiasi configurazione che accetta per stato finale

Nota. Per **$w \text{ non in } L(M)$** la TM M può:

- o **bloccarsi** prima o poi in uno stato non finale
- o **continuare per sempre** a computare senza mai raggiungere uno stato finale

← In entrambi i casi, la TM non accetta la stringa w

Un linguaggio L è Ricorsivamente Enumerabile se esiste una Macchina di Turing M che lo riconosce.

Comportamento: Per ogni stringa di input w :

- Se $w \in L$ (la stringa è nel linguaggio): M termina nello stato di accettazione ($q \in F$).

- Se $w \notin L$ (la stringa non è nel linguaggio): M non termina nello stato di accettazione. Può Rifiutare (terminare in uno stato non-finale di blocco) oppure entrare in un loop infinito (non terminare mai).

Nota: La proprietà cruciale che distingue gli LRE dai Linguaggi Ricorsivi è il comportamento sulla non-appartenenza.

Linguaggi Ricorsivamente Enumerabili

- Tali linguaggi vengono detti: **linguaggi ricorsivamente enumerabili**
 - Perché? Il termine venne inventato prima delle TM e fa riferimento ad una diversa nozione di calcolo basato su funzioni

Nota. La **computazione** della TM:

- **termina** per input **$w \in L(M)$**
- **può non terminare** per input **$w \notin L(M)$**

Problema di stabilire $w \in L(M)$: **semi-decidibile**

In altre parole, se l'MT tace, non possiamo sapere se sta ancora calcolando (e presto accetterà) o se è entrata in un loop infinito.

Un linguaggio L è Ricorsivo se e solo se esiste una Macchina di Turing M tale che:

- Se $w \in L$: M si ferma nello stato di accettazione.
- Se $w \notin L$: M si ferma nello stato di rifiuto (o si blocca in uno stato non-finale).
- Garantito che M termina sempre (si ferma in un tempo finito) per ogni possibile stringa di input w .

Quindi se TM esiste per Linguaggi Ricorsivi, allora esiste anche per Linguaggi ricorsivamente enumerabili, ma non viceversa (Esistono linguaggi che sono Ricorsivamente Enumerabili ma non sono Ricorsivi. Questi sono i linguaggi per cui esiste un Riconoscitore (una MT che accetta $w \in L$: semi decidibile), ma non un Decisore (una MT che si ferma su tutti gli input: decidibile sempre).)

I Linguaggi Ricorsivi sono un sottoinsieme stretto dei Linguaggi Ricorsivamente Enumerabili e corrispondono alla classe dei problemi che sono decidibili.

Linguaggi Ricorsivi

- Un **algoritmo** è una TM che **sicuramente termina** (cioè termina per ogni input)
- Se $L = L(M)$ per **una qualche** TM M che è un algoritmo, diciamo che L è un **linguaggio ricorsivo**
 - Perché? Vedi la risposta del lucido precedente....

Il punto chiave è che i Linguaggi Liberi dal Contesto non solo sono riconoscibili, ma sono anche decidibili (cioè, Ricorsivi).
Questo fa riferimento a un algoritmo chiamato Algoritmo di Parsing (Analisi Sintattica), in particolare quelli che lavorano con grammatiche in Forma Normale di Chomsky (CNF).
La Proprietà Chiave: Se una Grammatica Libera dal Contesto è in CNF, qualsiasi stringa w di lunghezza n (dove $n = |w|$) che può essere generata dalla grammatica richiede esattamente $2n-1$ passi di derivazione.
Garantire la Terminazione: Poiché il numero di alberi di derivazione di una data altezza è finito, e la stringa w può essere generata solo da alberi di altezza finita e limitata, è possibile costruire una Macchina di Turing Decisore che:
Prova in modo sistematico tutti i possibili alberi di derivazione con n foglie. Poiché il numero di prove è finito, $M_{\{Dec\}}$ termina sempre. Se trova un albero che genera w , Accetta. Se esaurisce tutti gli alberi possibili senza trovare w , Rifiuta.
Conclusione: La garanzia di terminazione rende i CFL Linguaggi Ricorsivi.

Esempio: Linguaggi Ricorsivi

- Ogni CFL è un linguaggio ricorsivo.
 - Basta usare l'algoritmo che, data w , prova tutti gli alberi sintattici con $|w|$ foglie
- ...in altri termini, ogni linguaggio per cui sia possibile in modo automatico **verificare se una data stringa vi appartiene oppure no** è ricorsivo
 - Problema di stabilire w in $L(M)$: **decidibile**

Se esiste un metodo finito garantito (un algoritmo) per risolvere un problema (decidere l'appartenenza al linguaggio), allora quel linguaggio è Ricorsivo. Questa classe rappresenta l'insieme massimo di problemi che i nostri computer possono risolvere in modo completo ed esauriente (fornendo una risposta definitiva e terminando sempre). I problemi che cadono al di fuori di questa classe sono intrinsecamente irrisolvibili.

Altro su Macchine di Turing

- “Trucchi” di programmazione
- Restrizioni
- Estensioni
- Proprietà di chiusura

Osservazione

- All'inizio, le TM non sembrano molto potenti
 - Ma sono veramente in grado di calcolare tutto quello che i calcolatori calcolano?
- Vedremo “**trucchi**” di **programmazione** ed **estensioni di TM** per convincerci che queste possono simulare calcolatori reali

La Macchina di Turing a Tracce Multiple è una tecnica fondamentale per dimostrare che, pur mantenendo un unico nastro fisico, la MT può gestire strutture dati più complesse.

①

Trucco di programmazione: Tracce multiple

La Macchina di Turing a Tracce Multiple è un modo per codificare più informazioni per cella senza alterare la definizione formale della MT standard.

- Consideriamo simboli del nastro che sono **vettori di k elementi**
- Ogni elemento viene scelto all'interno di un alfabeto finito
- E' come se il nastro avesse k tracce
- Simboli in **input**: hanno **un solo elemento** del vettore diverso da blank

Alfabeto del Nastro Modificato (Gamma'): Invece di usare l'alfabeto Gamma della MT standard, definiamo un nuovo alfabeto del nastro Gamma' che è il prodotto cartesiano degli alfabeti che vorremmo su ogni traccia.

Codifica: Se l'input $w=w_1w_2\dots w_n$ usa l'alfabeto Sigma, lo codifichiamo sul nuovo nastro T' in modo che solo la prima traccia contenga l'input, mentre le altre tracce contengano il simbolo blank (B) specifico per quell'alfabeto.

Esempio: tracce multiple

Rappresenta
l'input 0

q

Per avere il carattere Blank, devo avere Blank su tutte le tracce

Rappresenta
il blank

	0	X	B	
	B	Y	B	
	B	Z	B	

Rappresenta il simbolo [X,Y,Z]

②

Trucco di programmazione: Marcature

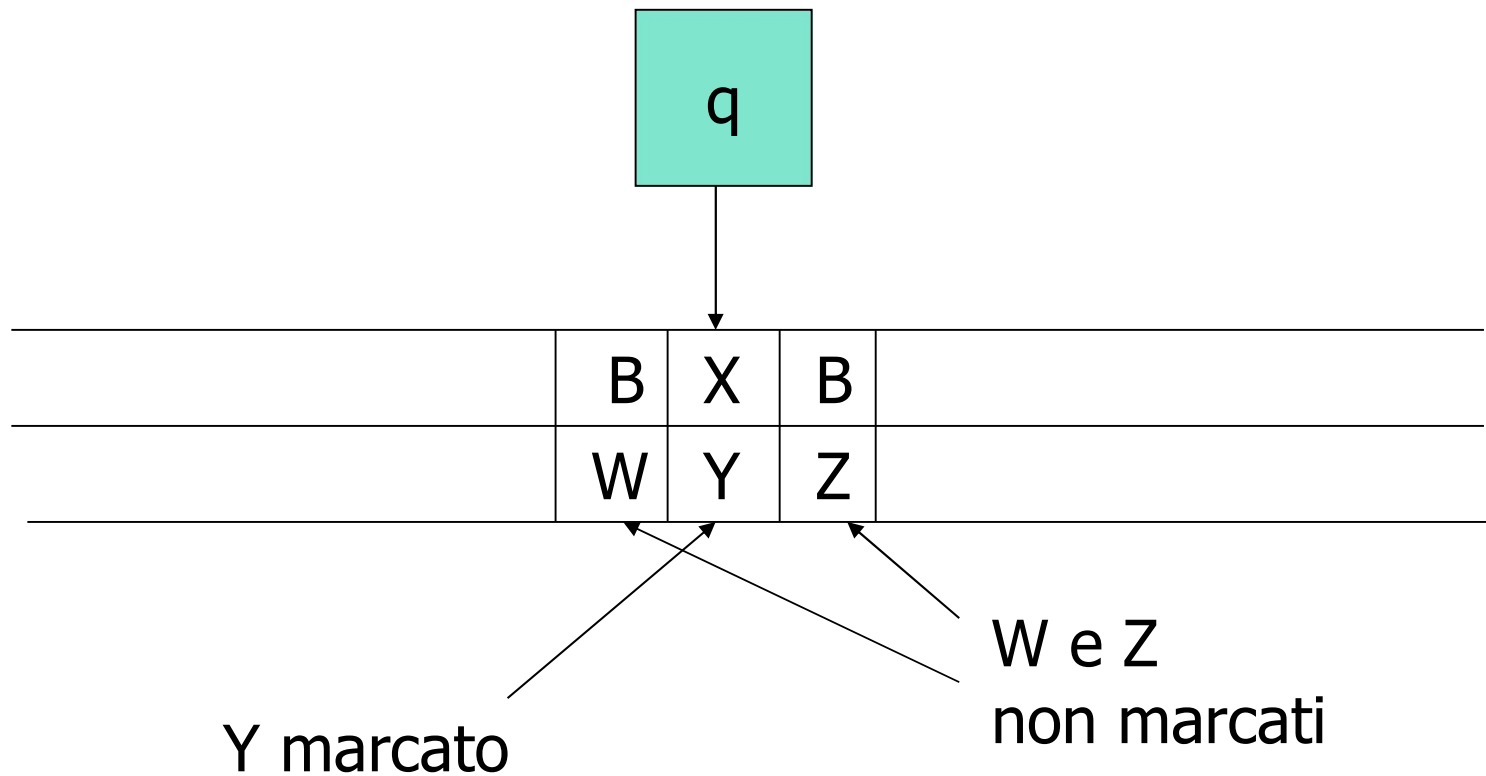
- Una traccia potrebbe essere usata per **marcare** alcune posizioni

Questa tecnica sfrutta una delle tracce per memorizzare informazioni di controllo, proprio come un puntatore o una variabile booleana in un linguaggio di programmazione.

- Quasi tutte le celle contengono il blank in questa traccia, ma alcune contengono un **simbolo speciale**

Simboli di Marcatura: L'alfabeto di questa traccia conterrà principalmente il simbolo Blank (B) ed un piccolo insieme di simboli speciali (ad esempio, X1, X2,...) che servono da marcatori. Funzione: La marcatura serve a "ricordare" una posizione specifica sul nastro mentre la testina si sposta altrove.

Esempio: Marcatura



Anche questa estensione non aumenta la potenza computazionale (espressività) della TM. Poiché l'insieme $S \times M$ delle piccole memorie è finito, l'insieme degli stati Q' (il prodotto cartesiano) è anch'esso finito (se Q è finito). La TM composita M' soddisfa tutti i requisiti formali di una TM standard.

L'uso di Stati con "Piccole Memorie" è un'altra tecnica di programmazione che dimostra come le limitazioni formali della Macchina di Turing (MT) possano essere aggirate, ancora una volta, senza aumentarne l'espressività.

③

Trucco di programmazione: Stati con “piccole memorie”

- Anche lo **stato** può essere un **vettore**
- Il primo elemento è lo “**stato di controllo**”
- Altri elementi contengono simboli presi da un alfabeto finito

- Stato Composto (Q'): Invece di avere un insieme di stati Q , definiamo un nuovo insieme di stati Q' che è il prodotto cartesiano dell'insieme di stati originale Q e l'insieme delle "piccole memorie" M .
- La Memoria: M è un insieme finito di simboli (o valori booleani, ecc.). Ad esempio, M_1 potrebbe essere l'alfabeto del nastro Gamma, e M_2 potrebbe essere $\{0, 1\}$.
- Significato: Se una TM si trova nello stato composito $q' = (q_{\text{controllo}}, \text{simbolo } X, \text{flag } 1)$, il suo comportamento non dipenderà solo dal simbolo letto dal nastro, ma anche dai valori memorizzati internamente (X e 1).

Esempio: uso di questi “trucchi”

- Questa TM semplicemente **copia l' input** (stringa binaria) **infinite volte** (Quindi, non accetta mai: il linguaggio di questa TM é vuoto)
- Stati di controllo:
 - q: marca la posizione e memorizza il simbolo letto
 - p: va a destra, continuando a ricordarsi il simbolo letto, finché non si trova un blank, dove depositerà il simbolo
 - r: va a sinistra fino alla marcatura

Esempio – (2)

- Gli **stati** hanno forma $[x,Y]$, dove:
 - x è q , p , oppure r
 - Y è 0 , 1 , oppure B
 - 0 e 1 usati solo in caso x sia p
- I **simboli** del nastro hanno forma $[U,V]$
 - U è X (la “marcatura”) oppure B
 - V è 0 , 1 (i simboli di input) oppure B
 - $[B,B]$ è il blank della TM; $[B,0]$ e $[B,1]$ sono gli input

← La funzione di transizione

- Convenzione: a e b sono simboli dell'alfabeto di input (cioè 0 oppure 1)
- $\delta([q, B], [B, a]) = ([p, a], [X, a], R)$
 - Nello stato q , copia il simbolo in input sotto la testina (cioè a) nello stato
 - Marca la posizione letta
 - Va allo stato di controllo p e muove a destra

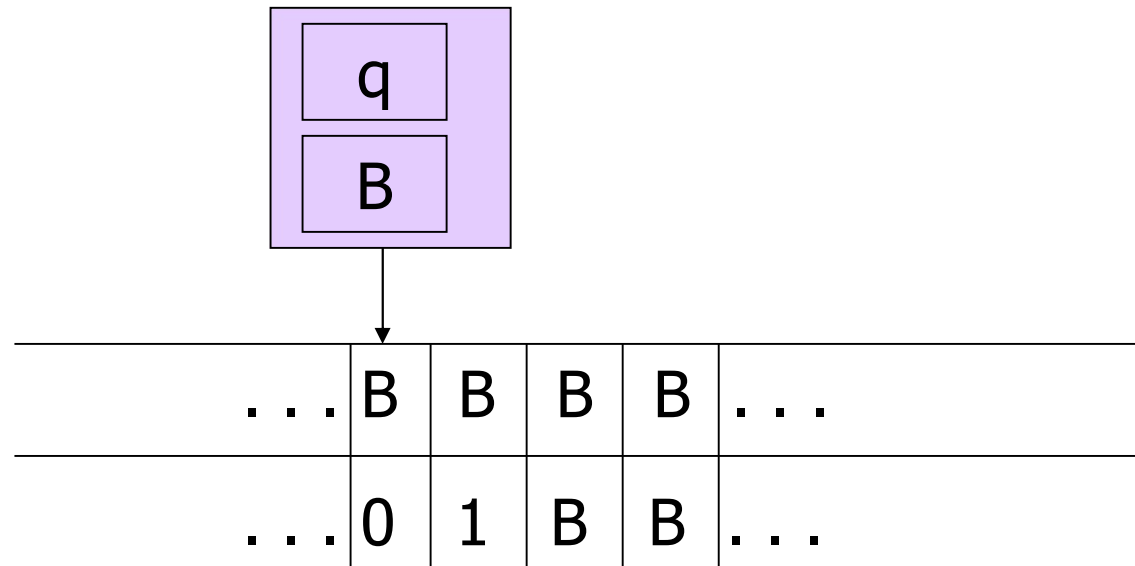
Funzione di transizione – (2)

- $\delta([p,a], [B,b]) = ([p,a], [B,b], R)$
 - Nello stato p , va a destra, in attesa di trovare un simbolo blank (non un blank nella sola traccia di marcatura)
- $\delta([p,a], [B,B]) = ([r,B], [B,a], L).$
 - Quando trova B, lo rimpiazza con il simbolo memorizzato nello stato (cioè a)
 - Entra nello stato r e va a sinistra

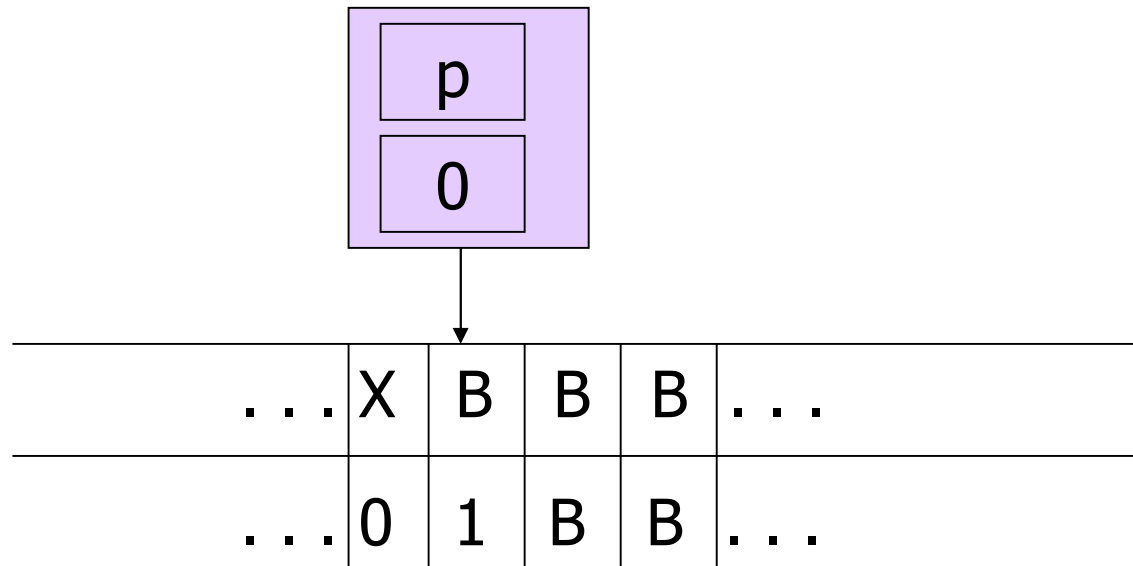
Funzione di transizione – (3)

- $\delta([r,B], [B,a]) = ([r,B], [B,a], L)$
 - Nello stato r , va a sinistra, alla ricerca della marcatura
- $\delta([r,B], [X,a]) = ([q,B], [B,a], R)$
 - Quando la marcatura viene trovata, entra nello stato q e va a destra
 - Rimuove la marcatura
 - Lo stato q inserirà una marcatura e si ripeterà il ciclo

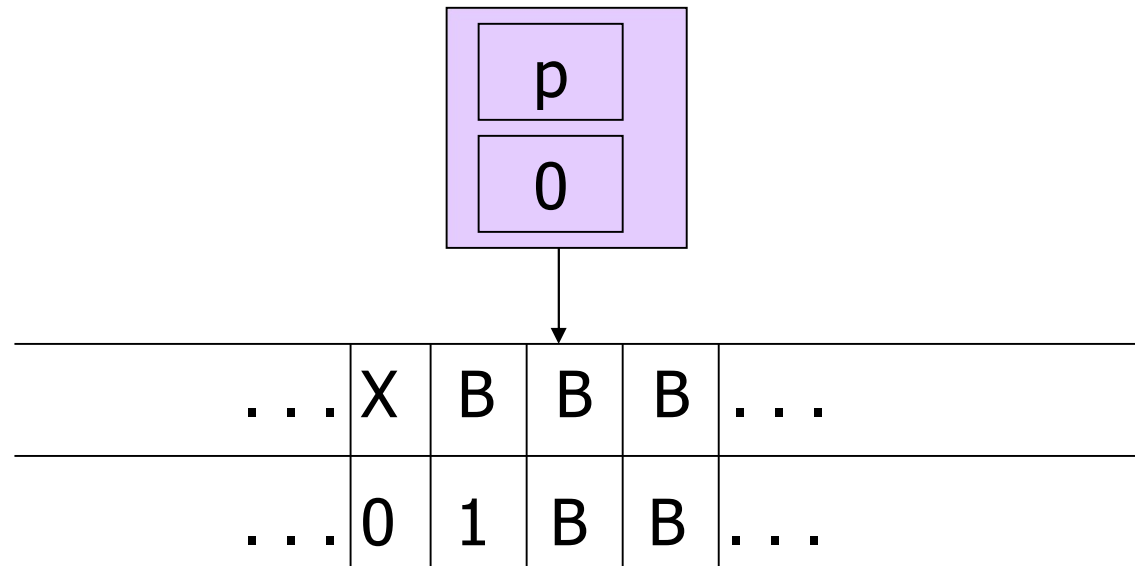
Simulazione della TM



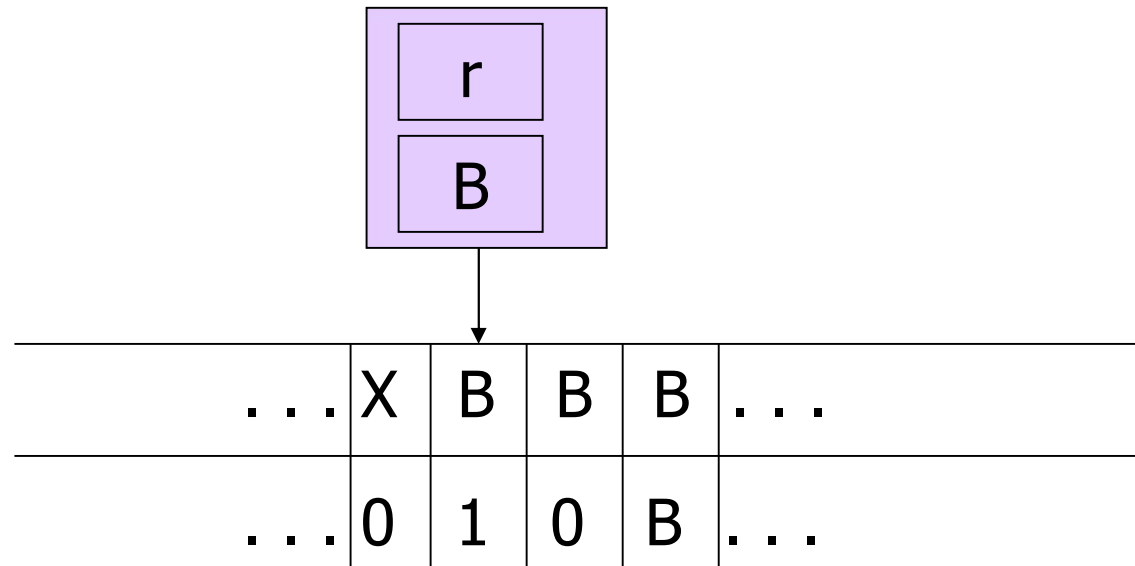
Simulazione della TM



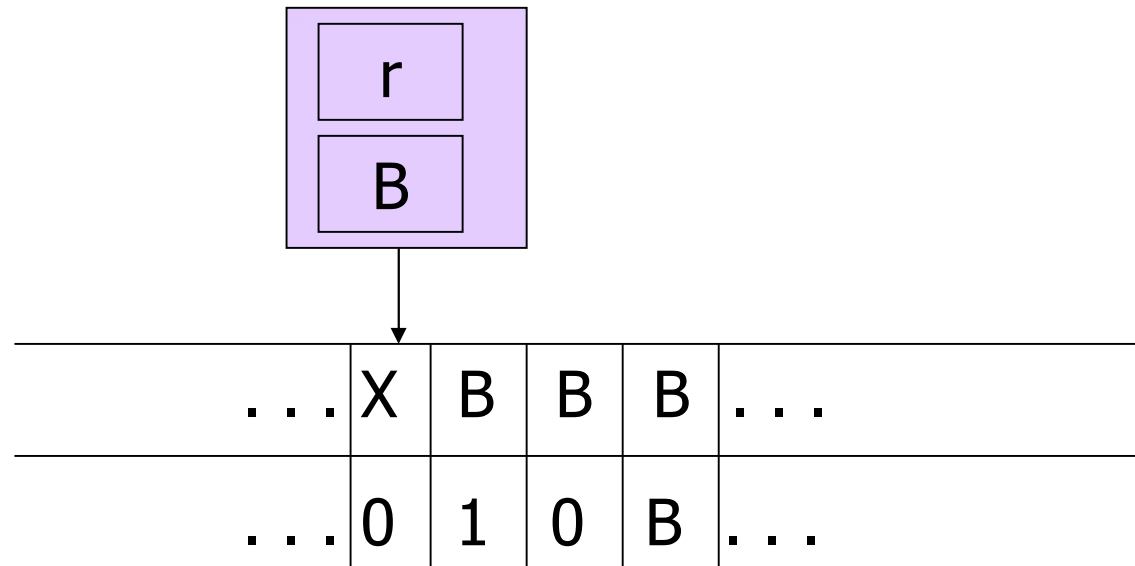
Simulazione della TM



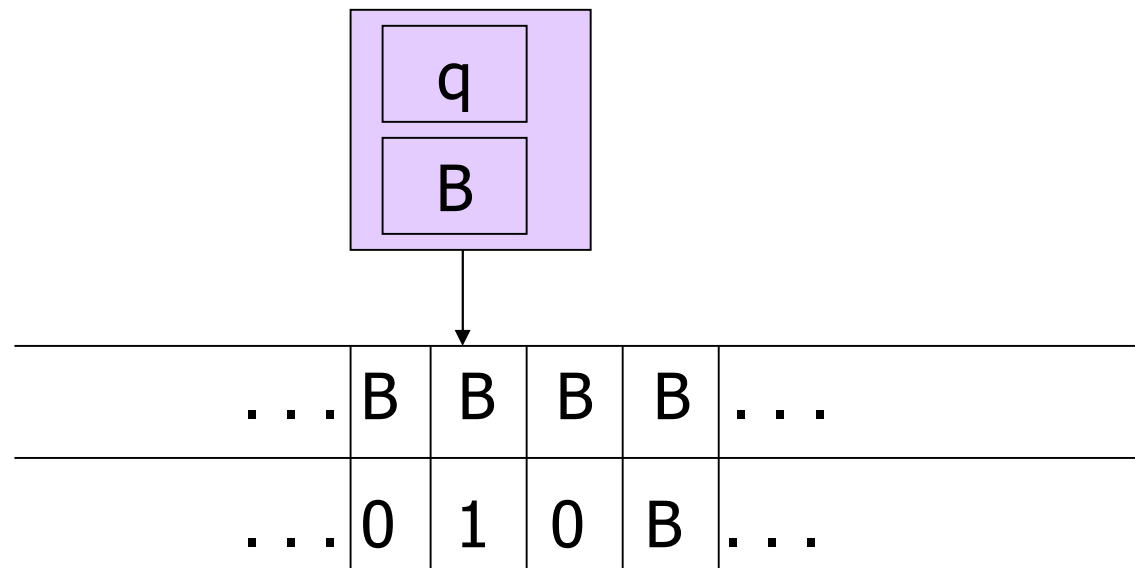
Simulazione della TM



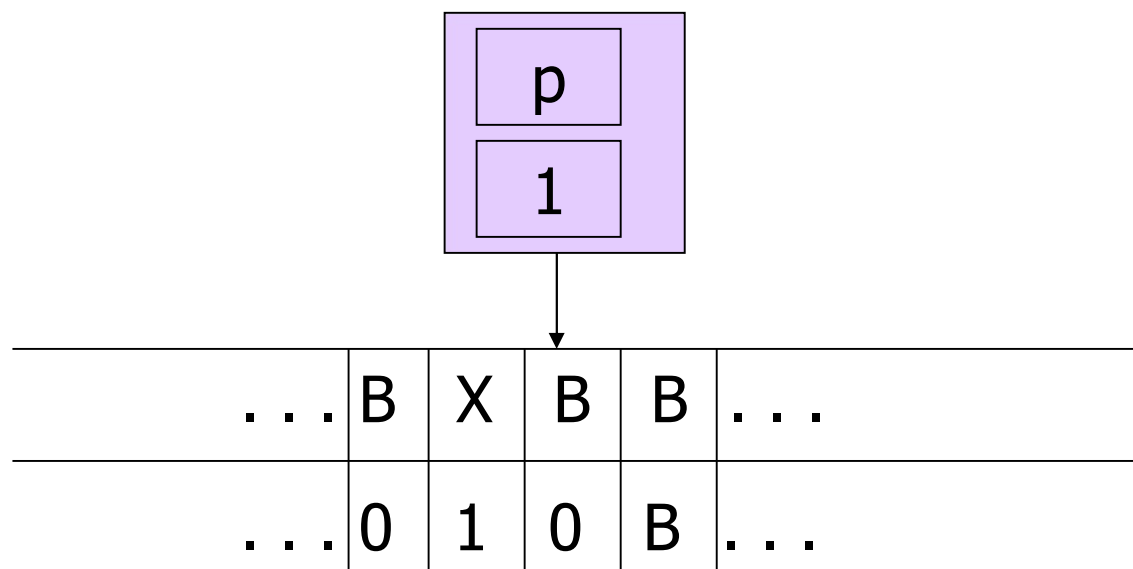
Simulazione della TM



Simulazione della TM



Simulazione della TM



Restrizioni alla TM

La nuova Macchina di Turing M' (il simulatore) ha un nastro semi-infinito (che parte dalla posizione 0 e si estende solo a destra) e sfrutta il trucco delle tracce multiple per codificare il nastro negativo.

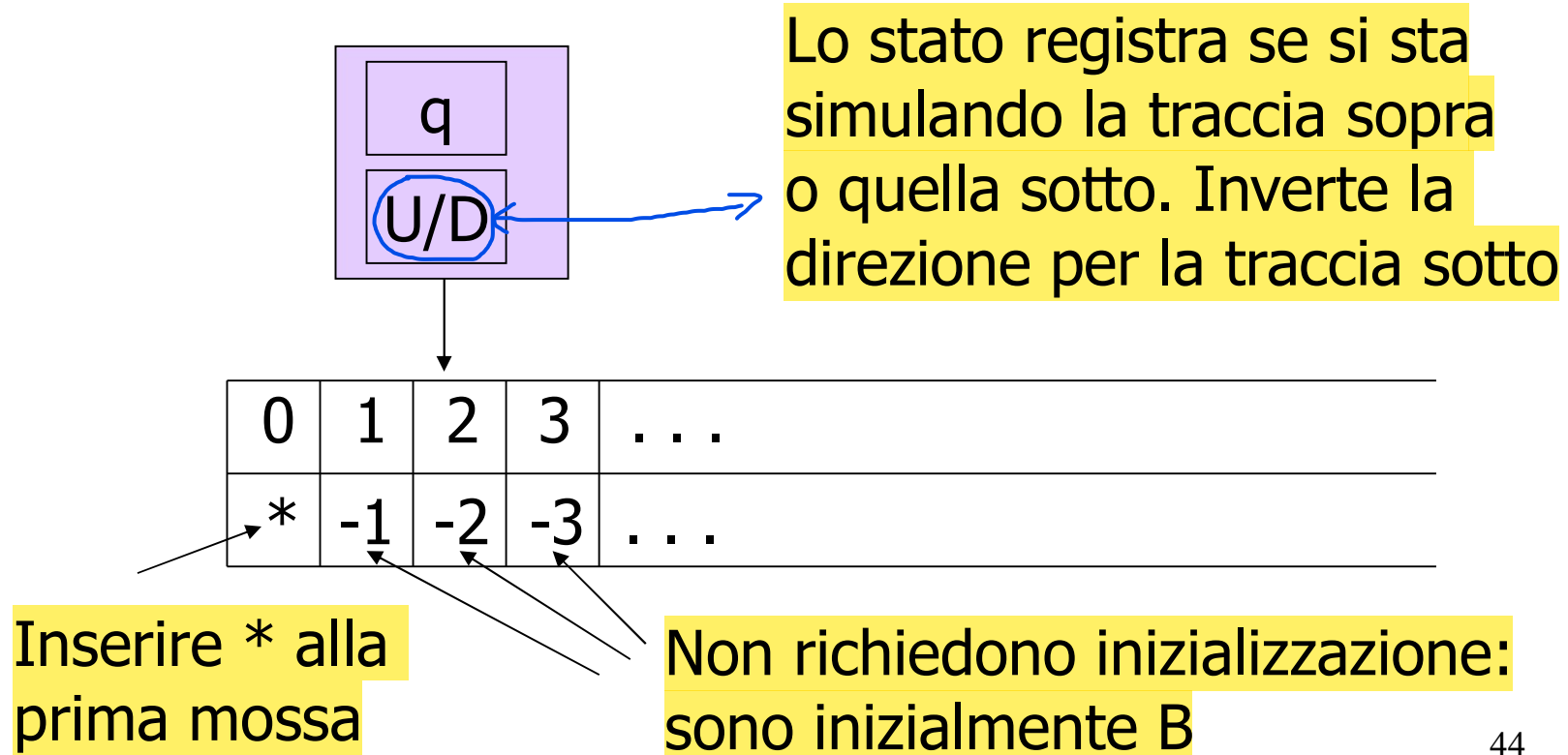
Nastro semi-infinito

Il Modello da Simulare: Nastro Infinito su Due Lati: Le celle sono indicizzate da $\dots, -2, -1, 0, 1, 2, \dots$. La testina può muoversi a sinistra e a destra senza limiti.

- Possiamo assumere che la TM non si muova **mai a sinistra della posizione iniziale**
- Indichiamo con 0 tale posizione; con $1, 2, \dots$ le posizioni a destra; con $-1, -2, \dots$ le posizioni a sinistra
- La nuova TM usa **due tracce**:
 - Sopra contiene i simboli in posizione $0, 1, 2, \dots$
 - Sotto i simboli in posizione $-1, -2, \dots$

Questa simulazione dimostra che una MT a nastro infinito bilaterale non è più potente di una MT a nastro semi-infinito. Ogni linguaggio Ricorsivamente Enumerabile riconosciuta dal modello bilaterale è riconoscibile anche dal modello monolaterale. Questa è un'altra importante dimostrazione che il modello di Turing è robusto e che le sue variazioni non ne aumentano la potenza teorica.

Simulazione del nastro infinito con quello semi-infinito



Due Pile Simulano un Nastro: Il punto chiave è che la capacità di una MT di muovere la testina a sinistra è l'unica caratteristica che la distingue dai PDA e che le conferisce la potenza computazionale (espressività) completa.

1. Simulazione del Nastro con Due Pile

- Un nastro infinito può essere concettualmente diviso in due sezioni, separate dalla testina:

- Pila Sinistra (Stack 1): Contiene tutti i simboli alla sinistra della testina. Poiché l'accesso avviene dalla cima, la cima della Pila 1 è il simbolo immediatamente a sinistra della testina.

- Pila Destra (Stack 2): Contiene tutti i simboli a destra della testina, incluso il simbolo sotto la testina (che è la cima della Pila 2).

Ulteriori restrizioni

Poiché il PDA utilizza una pila, e l'aggiunta di una seconda pila consente di simulare perfettamente tutte le operazioni di un nastro di Turing (incluso il movimento a sinistra e a destra), la Macchina di Turing è formalmente equivalente a un Automa con Due Pile (Two-Stack PDA).

- **Due stack** possono simulare un nastro
 - Uno contiene le **posizioni alla sinistra della testina**; l'altro le **posizioni alla destra**
- **Conclusione:** in sostanza, la differenza fra PDA e TM, è che **la TM ha due pile** invece che una sola!

L'aggiunta di una seconda pila aumenta la potenza da un modello limitato (il PDA) a un modello che è computazionalmente universale (la TM), rendendola capace di riconoscere linguaggi che non sono CFL.

Nonostante la Macchina di Turing a Nastri Multipli possa eseguire calcoli molto più velocemente (è più efficiente) della Macchina di Turing a Nastro Singolo, la sua potenza computazionale (espressività) teorica rimane la stessa.

La TM a Nastri Multipli è una delle estensioni.

Turing Machine Multinastro

Struttura del Modello

- k Nastri e k Testine: Invece di un singolo nastro, la macchina ne ha k distinti, ognuno con la sua testina indipendente.
- Controllo Finito Unico: C'è un solo stato di controllo (q in Q) per l'intera macchina.

- Permettiamo alle TM di avere **k nastri** (per un qualsiasi k) e **k testine**

- Le mosse della TM dipendono dallo stato e dal simbolo sotto la testina di ogni nastro

- In una mossa, la TM può cambiare stato, scrivere simboli sotto le testine, e muovere **ogni testina indipendentemente**

la macchina può leggere, scrivere e muovere su tutti e k i nastri in parallelo, in un singolo passo di calcolo.

$T(n)$ è il numero massimo di mosse che la Macchina di Turing M esegue per un input di lunghezza n (dove $n = |w|$).

la doppia scansione è necessaria per separare l'operazione di lettura (raccolta dei dati: marcatura e dato associato) dall'operazione di aggiornamento dei dati e dei marcatori, entrambe eseguite in parallelo da TM multi-tape.

La Macchina di Turing a nastri multipli è equivalente alla MT a nastro singolo, ma non altrettanto efficiente. Se la TM multi-tape termina in $O(T(n))$ passi (dove n è la lunghezza dell'input), la TM single-tape richiederà un tempo molto più lungo, tipicamente $O((T(n))^2)$. Questo perché, per ogni passo della TM multi-tape, la TM single-tape deve scansionare l'intera area attiva del nastro due volte, e la dimensione di quest'area cresce con il tempo di calcolo.

La MT a nastro singolo simula l'azione parallela della MT a k nastri in una sequenza di mosse più lenta, ma equivalente.

Dimostrare l'equivalenza tra la Macchina di Turing a k nastri e la Macchina di Turing a nastro singolo

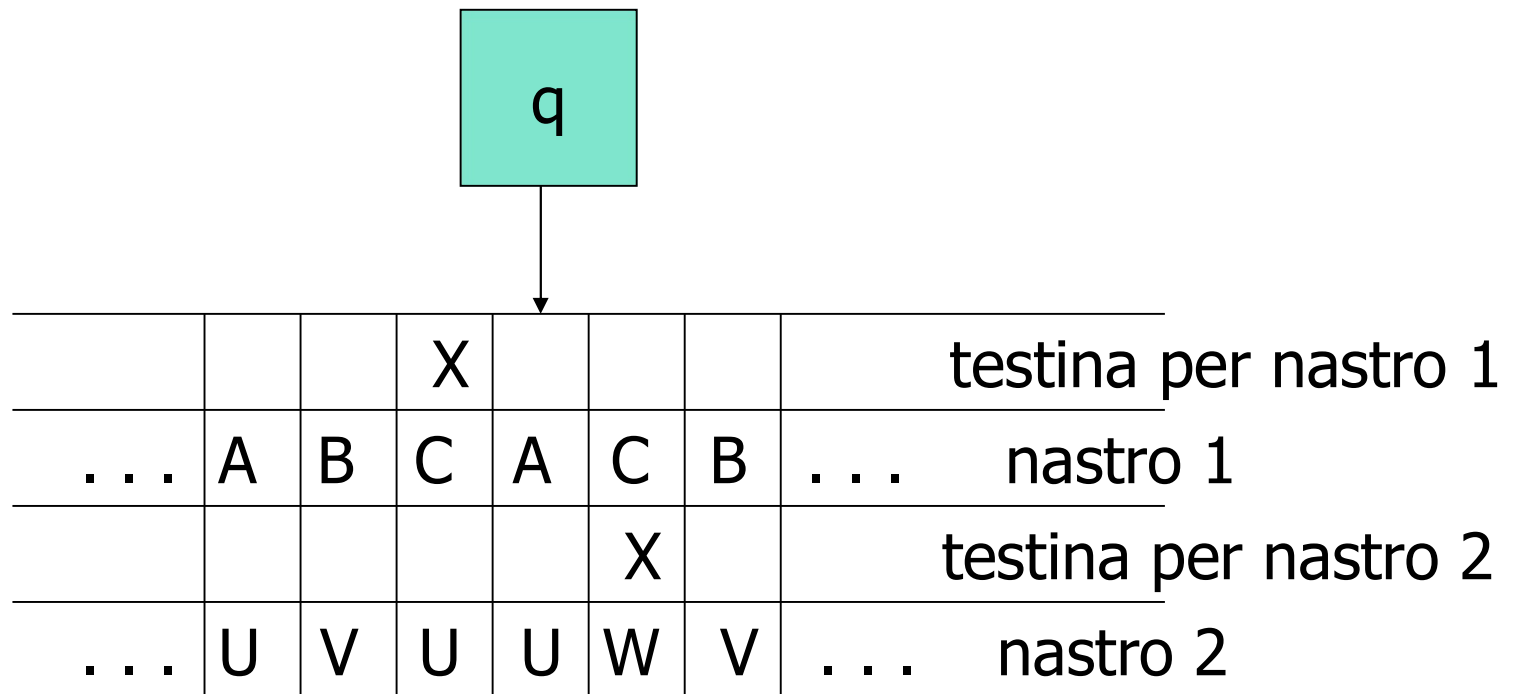
Simulare k nastri con uno solo

- Usare **$2k$ tracce**
- Ogni **nastro** è rappresentato da una **traccia**
- La posizione della corrispondente testina è **marcata** in una **traccia aggiuntiva**
Ogni traccia aggiuntiva contiene un marcatore speciale in un'unica cella, indicando la posizione della testina corrispondente
- Ci si muove avanti/indietro simulando i passi di ogni singola testina
 - Una **passata** simula una **mossa**

Una singola mossa della TM multi-tape (che avviene in un solo passo su tutti i nastri in parallelo) viene simulata da una sequenza di mosse di TM single-tape.

Immagine della simulazione

Simulo di avere TM multi-tape con k nastri
tramite una TM single-tape con 2k tracce



Nelle Macchine di Turing Deterministiche (DTM), la funzione di transizione delta è una funzione: per ogni stato e simbolo letto, esiste al più una mossa successiva definita. Nella NTM, delta è una relazione o una funzione a valori di insieme: ciò significa che, dato lo stato q e il simbolo letto Z, ci possono essere zero, una o più triple di mosse possibili.

Un'altra estensione della TM

TM Nondeterministiche

- Permettere alle TM di avere una scelta fra varie possibili mosse
- Ogni **scelta** è una **possibile tripla** nuovo stato - simbolo da scrivere - direzione (le TM deterministiche hanno al più^{max} una tripla)
- La TM accetta l'input se esiste una **possibile sequenza di scelte** che porta ad uno stato accettante

Equivalenza di Potenza (Il Risultato Fondamentale)
La Macchina di Turing Non Deterministica non aumenta la potenza computazionale (espressività) teorica della MT.

Teorema di Equivalenza: Per ogni Macchina di Turing Non Deterministica (N) che riconosce un linguaggio L, esiste una Macchina di Turing Deterministica (D) che riconosce esattamente lo stesso linguaggio L.

Il non determinismo non significa che la macchina esegue tutte le scelte contemporaneamente (anche se è utile immaginarlo così), ma che se esiste almeno un percorso di calcolo (un ramo dell'albero di calcolo) che porta allo stato di accettazione, allora la stringa di input viene accettata. Albero di Calcolo: Per ogni input w, la computazione di una NTM genera un albero di calcolo in cui ogni nodo è una configurazione della macchina e i rami sono le possibili scelte di delta. Se almeno una foglia di questo albero è uno stato di accettazione, la NTM accetta w.

Nonostante il non determinismo, la Macchina di Turing Non Deterministica (NTM) può essere simulata da una Macchina di Turing Deterministica (DTM), mantenendo l'equivalenza di potenza.

La DTM utilizza il nastro, con tracce multiple per gestire i dati necessari per la simulazione:

- Nastro (Traccia Principale): Viene usata per memorizzare una sequenza di ID.
- Struttura Dati: Le ID vengono gestite come una coda (FIFO: chi prima entra, esce), fondamentale per la ricerca in ampiezza.

Simulare una NTM con una DTM

- La DTM mantiene sul proprio nastro una **coda di ID** della NTM
- Una **seconda traccia** è usata per
 - **Marcare la ID in testa alla coda**
 - Generare, una alla volta, le **successive ID di quella in testa alla coda**, per poi copiarle in fondo alla coda

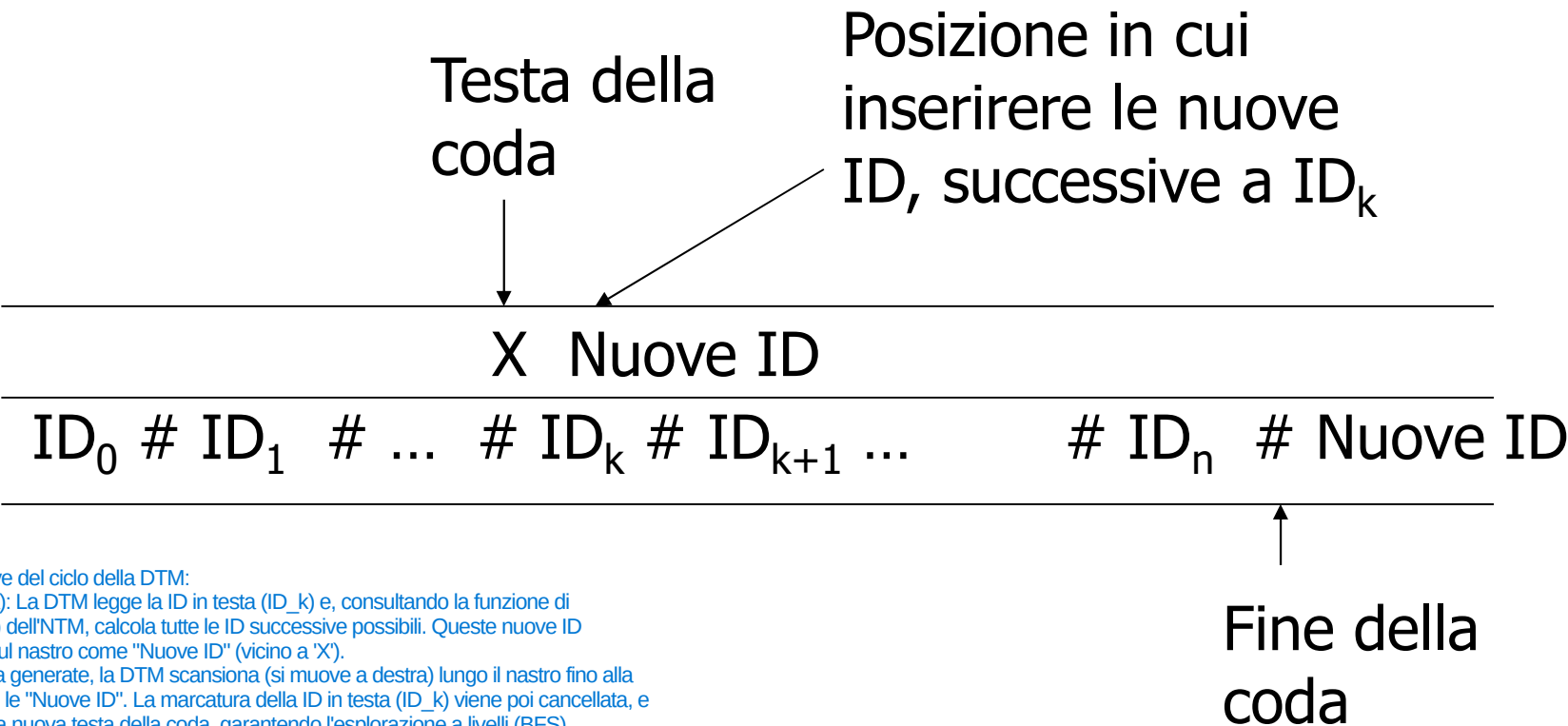


La ricerca in ampiezza (BFS) garantisce la terminazione se e solo se:

- La NTM accetta (trova un ramo accettante).
- La NTM non entra in loop.

- La sequenza di ID sul nastro della DTM agisce come una coda FIFO (First-In, First-Out), essenziale per la BFS:
- Testa della Coda (ID_k nell'esempio): Questa è la ID che la DTM sta elaborando in quel momento. Viene letta per generare le possibili ID successive.
 - La testina della DTM è posizionata sulla ID in testa (indicata da 'X' e "Testa della coda").
 - Fine della Coda (Dopo ID_n): Le nuove ID generate come risultato della simulazione di un passo della NTM vengono inserite in fondo alla coda ("Posizione in cui inserire le nuove ID, successive a ID_k ").

Immagine del nastro della DTM



La figura illustra i due passaggi chiave del ciclo della DTM:

- Lettura e Generazione (Sulla Testa): La DTM legge la ID in testa (ID_k) e, consultando la funzione di transizione non deterministica (delta) dell'NTM, calcola tutte le ID successive possibili. Queste nuove ID vengono scritte temporaneamente sul nastro come "Nuove ID" (vicino a 'X').
- Accodamento (In Fondo): Una volta generate, la DTM scansiona (si muove a destra) lungo il nastro fino alla fine della coda (dopo ID_n) e copia lì le "Nuove ID". La marcatura della ID in testa (ID_k) viene poi cancellata, e la ID successiva (ID_{k+1}) diventa la nuova testa della coda, garantendo l'esplorazione a livelli (BFS).

Questa visualizzazione conferma che la simulazione di una NTM con una DTM è un processo meccanico e deterministico, che si basa sull'uso del nastro come struttura dati (una coda) per organizzare l'esplorazione dell'albero di calcolo non deterministico.

Operazioni della DTM

- La DTM cerca la ID in testa alla coda
- Cerca lo stato della ID così da poter determinare le possibili mosse
- Se ci sono m possibili mosse, **crea m nuove ID**, una per ogni mossa, e le copia in fondo alla coda

Operazioni della DTM – (2)

- Dopo che tutte le ID sono state messe in fondo alla coda, si **sposta in avanti la marcatura in testa alla coda**
- Tuttavia, se si crea una **ID con stato accettante**, la DTM si ferma in uno stato accettante

Quali vantaggi dalle estensioni?

- Ecco i vantaggi...
- Quando discutiamo di **costruzioni** che lavorano a partire da una TM, possiamo assumere che tale TM sia semplice
 - Cioè una TM deterministica con un nastro semi-infinito
- Ma la TM che costruiamo può avere più nastri, essere nondeterministica, ...

Proprietà di chiusura per linguaggi Ricorsivi e RE

- Entrambi chiusi rispetto a unione, intersezione, concatenazione, stella di Kleene e reverse
- Ricorsivi chiusi rispetto a complemento e differenza

M_{unione} prende l'input w (che si trova sul Nastro 1 all'inizio).
 M_{unione} esegue una scansione per copiare w dal Nastro 1 al Nastro 2. (Questo passo è facile e deterministico, ed è il motivo per cui è utile avere nastri multipli).

Unione

M_{unione} ora simula M_1 e M_2 in modo interleaved sulle rispettive tracce. La funzione di transizione di M_{unione} alterna i passi di simulazione:

- Passo 1: Simula un passo di M_1 sul Nastro 1.
- Passo 2: Simula un passo di M_2 sul Nastro 2.
- Passo 3: Simula un passo di M_1 sul Nastro 1.
- e così via...

- Sia $L_1 = L(M_1)$ e $L_2 = L(M_2)$
- Assumiamo che M_1 e M_2 siano TM con un solo nastro semi-infinito
- Costruiamo una TM con 2 nastri che prima copia l'input sul secondo nastro e poi simula "in parallelo" le due TM (M_1 sul primo nastro, M_2 sul secondo nastro)

Costruiamo una nuova Macchina di Turing M_{unione} che riconoscerà $L_1 \cup L_2$.
Per semplificare la costruzione, utilizziamo una MT a due nastri:
Nastro 1: Verrà utilizzato per simulare M_1 .
Nastro 2: Verrà utilizzato per simulare M_2 .

L'operazione di unione garantisce che:
- Se $w \in L_1 \cup L_2$, M_{unione} troverà un'accettazione (da M_1 o M_2) e terminerà accettando.
- Se $w \notin L_1 \cup L_2$, M_{unione} non accetterà mai. La simulazione in parallelo è essenziale qui, perché se M_1 accetta ma M_2 entra in loop, la simulazione alternata M_{unione} troverà e segnalerà l'accettazione di M_1 prima che la simulazione di M_2 possa dominare il tempo di calcolo all'infinito.

Il punto di distinzione tra un linguaggio Ricorsivo e un linguaggio RE è la terminazione garantita della Macchina di Turing (MT) associata.

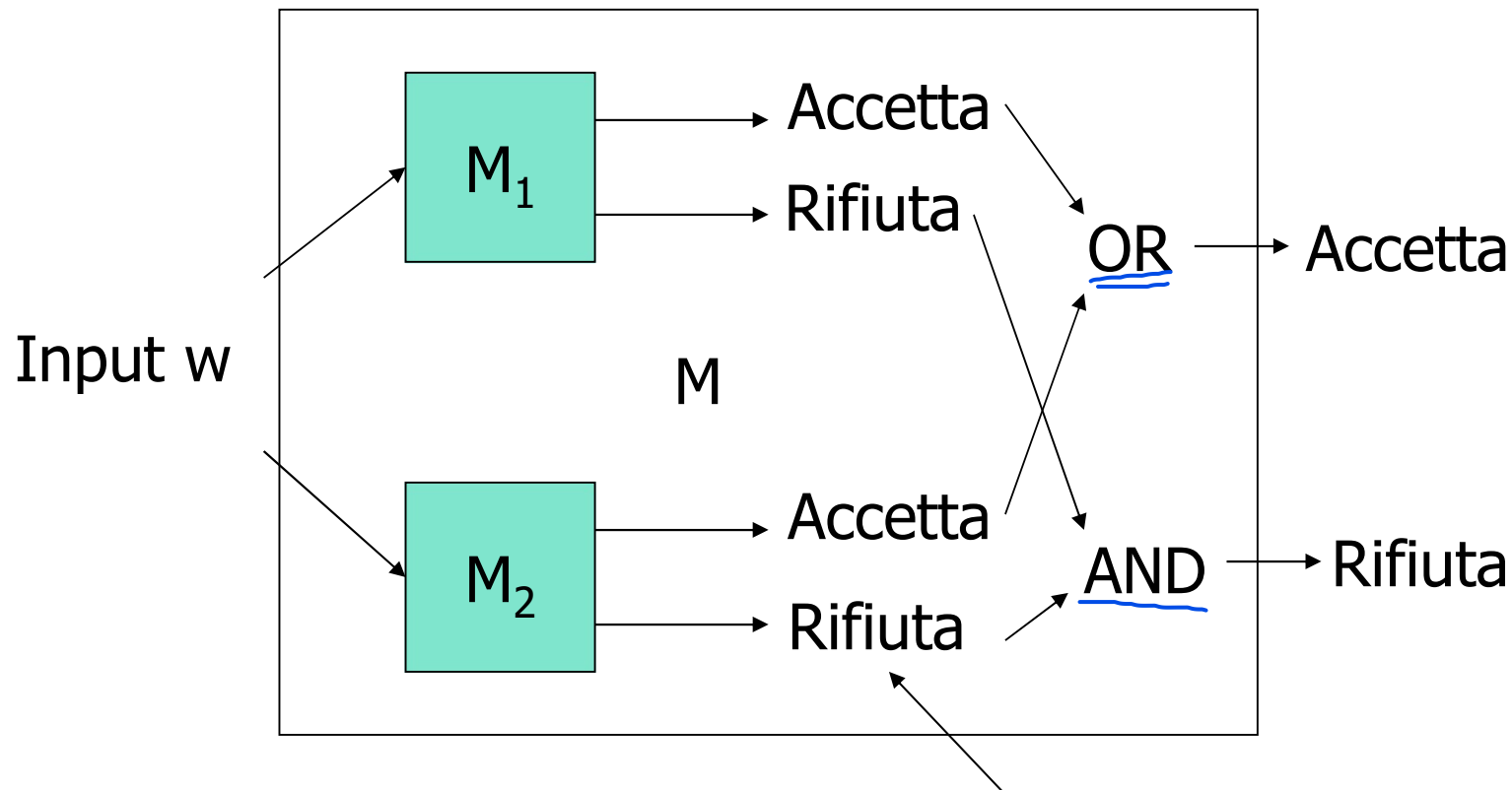
Unione – (2)

- Linguaggi **Ricorsivi**: se M_1 e M_2 sono entrambi algoritmi, allora M sicuramente termina entrambe le simulazioni (è quindi un algoritmo)
 - Accetta se almeno una accetta
- Linguaggi **RE**: accetta se almeno una accetta, ma potrebbe succedere che una o entrambe le TM non terminino mai (né si bloccano, né accettano)

Se né M_1 né M_2 accettano, entrambe terminano nel loro stato di rifiuto, e M_{unione} può essere progettata per entrare in uno stato di rifiuto, terminando così sempre.

In RE, TM può rifiutare

Immagine per Unione / Ricorsivi

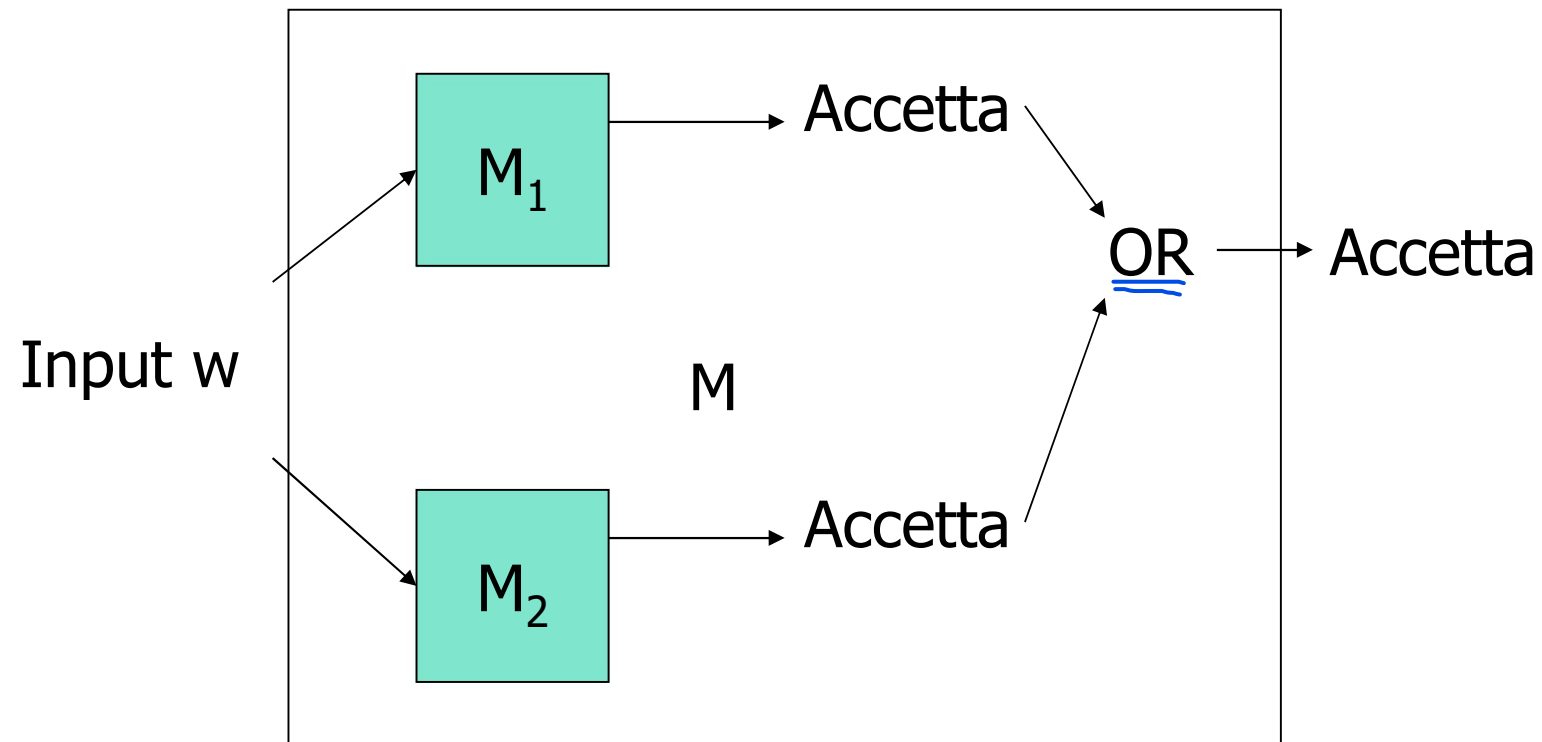


AND: devo attendere
entrambi gli input

OR: me ne basta uno

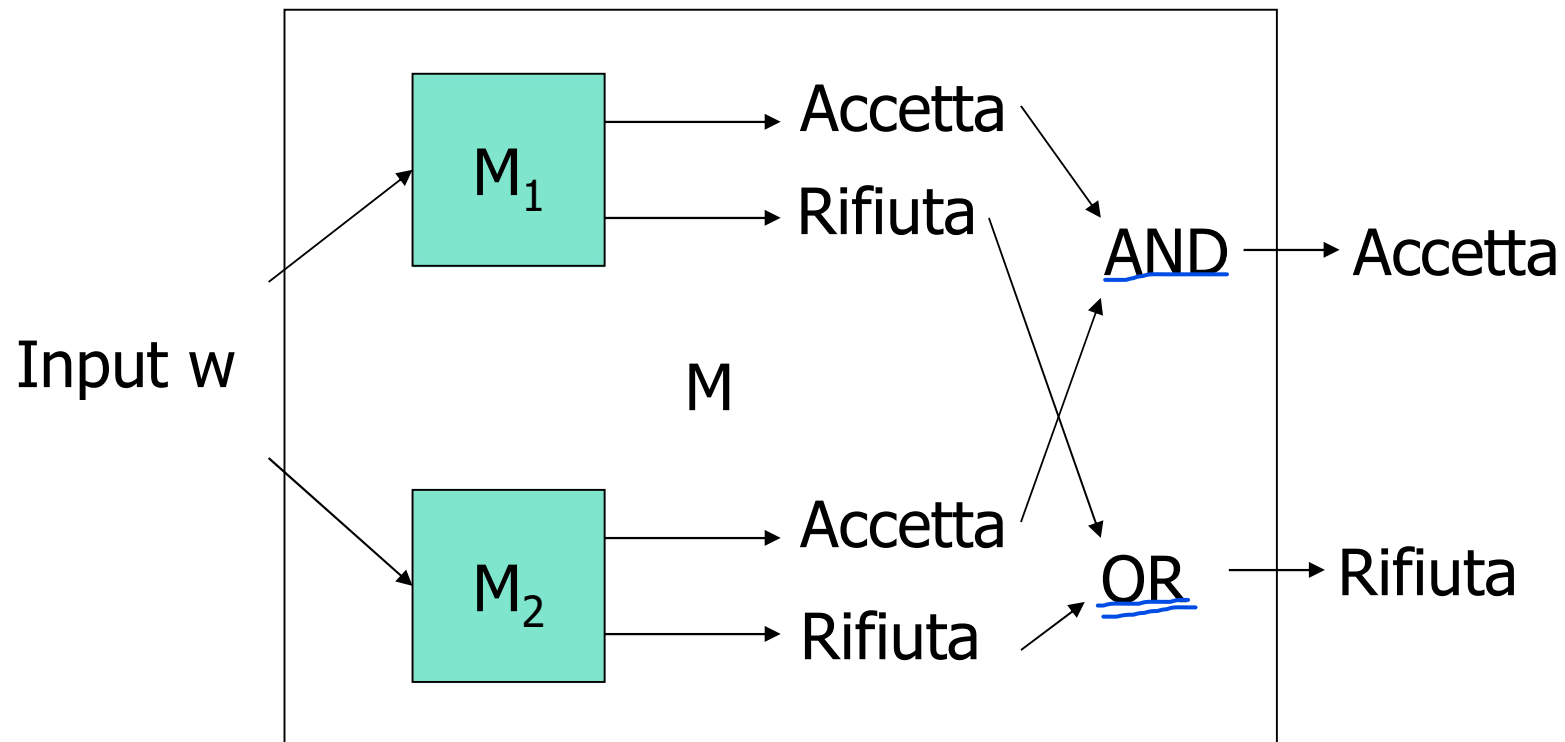
Ricorda: blocca
senza accettare

Immagine per Unione / RE

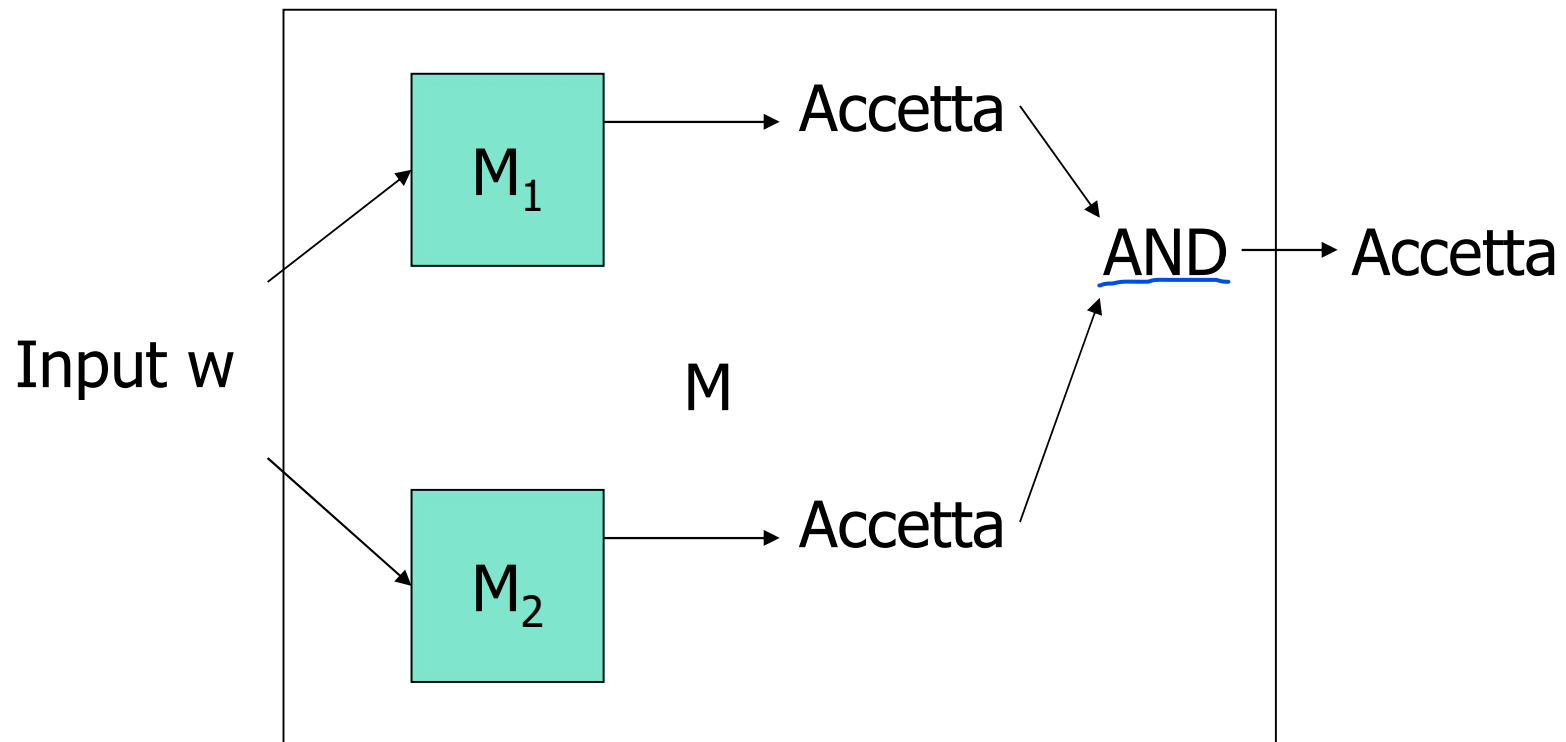


Dimostro allo stesso modo dell'Unione, ma inverto l'AND con l'OR e viceversa

Intersezione/Ricorsivi – Stessa Idea



Intersezione RE



Il complemento di un linguaggio L è l'insieme di tutte le stringhe che non appartengono a L .

Complemento

- Linguaggi **Ricorsivi**: la TM M si fermerà prima o poi
 - Accetta se M non accetta (cioè prendo la risposta di M e faccio l'inverso)
- Linguaggi **RE**: non si può fare la medesima costruzione!
 - per una stringa che non è in $L(M)$, M potrebbe non terminare mai (non si riesce quindi a farla andare in uno stato finale)

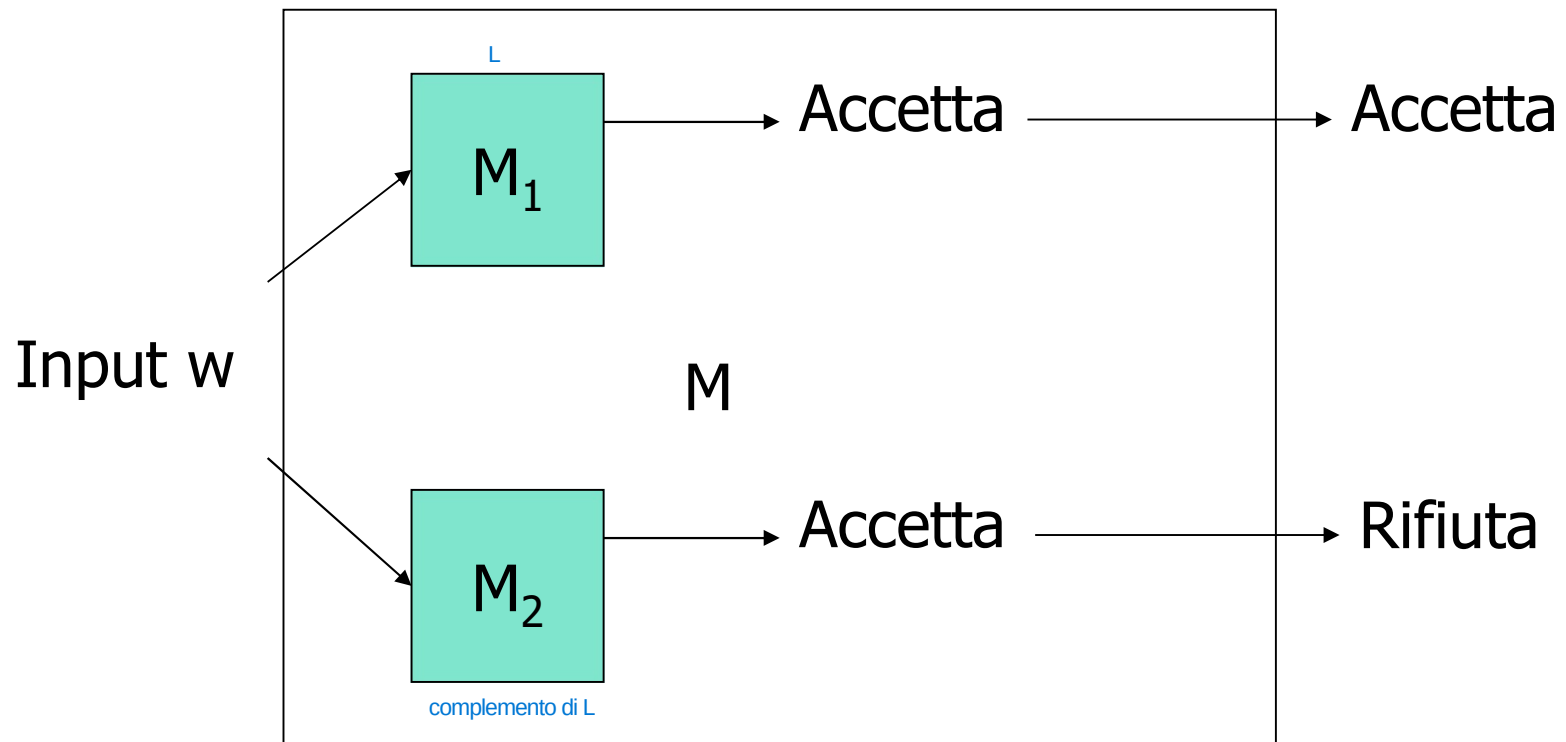
Se L è solo RE, la sua MT M può ciclare per $w \notin L$. Ciò significa che non esiste un algoritmo garantito che ci permetta di dire "NO, w non è in L ". Se non possiamo dire "NO" in tempo finito, non possiamo costruire una MT che riconosca il complemento di L .

Complemento di RE

La classe dei Linguaggi Ricorsivamente Enumerabili non è chiusa rispetto al complemento. In generale, se L è RE, il complemento di RE non è necessariamente RE.

- In generale il **complemento** di un linguaggio L **RE** può essere **non RE**
- **Teorema di Post**
Se L è **RE** ed il complemento di L è **RE** allora si ha che L è **Ricorsivo**
 - cioè, se L riconosciuto da una TM M_1 e **complemento di L** riconosciuto da una TM M_2 allora L è **Ricorsivo**

Linguaggio e suo complemento RE



Differenza

- Differenza tra L_1 ed L_2 è **intersezione** di L_1 con **complemento** di L_2 $L_1 \cap \bar{L}_2$
- Linguaggi **Ricorsivi**: chiusi rispetto a entrambe le operazioni di intersezione e complemento
- Linguaggi **RE**: se fosse chiuso rispetto a differenza lo sarebbe anche rispetto a complemento (basta prendere come L_1 l'insieme di tutte le stringhe) Linguaggi RE non sono chiusi rispetto a Differenza perché non sono chiusi rispetto a Complemento

- Test con Σ^* : Scegliamo un linguaggio molto semplice per L_1 : l'insieme di tutte le stringhe, Σ^* .
> Σ^* è banalmente Ricorsivo (e quindi RE), riconosciuto da una DTM che accetta immediatamente.
- La Sostituzione: Appliciamo l'operazione di Differenza: $\Sigma^* - L_2 = \Sigma^* \cap \bar{L}_2 = \bar{L}_2$
- La Conclusione Logica: Se la classe RE è chiusa rispetto alla Differenza, e Σ^* è RE, e L_2 è RE, allora anche $\Sigma^* - L_2 = \bar{L}_2$ deve essere RE.

Ma noi sappiamo, per il Teorema di Post, che non tutti i complementi di linguaggi RE sono RE (se lo fossero, tutti i linguaggi RE sarebbero Ricorsivi, il che è falso).
Risultato: L'assunzione è falsa. La classe dei Linguaggi Ricorsivamente Enumerabili non è chiusa rispetto alla Differenza.

Riconoscimento: Se $w \in L_1 L_2$, allora esiste almeno una divisione $w=xy$ tale che $x \in L_1$ e $y \in L_2$. Poiché M_{conc} è non deterministica, proverà (selezionerà) quel particolare ramo di calcolo corretto. Su quel ramo, M_1 accetterà x e M_2 accetterà y , e M_{conc} accetterà w .
Potenza: Poiché la Macchina di Turing a Nastri Multipli Non Deterministica è equivalente in potenza alla Macchina di Turing standard, questo dimostra che $L_1 L_2$ è RE.

$$L_1 L_2 = \{w \mid w = xy, \text{dove } x \in L_1 \text{ e } y \in L_2\}$$

Concatenazione / RE

- Siano $L_1 = L(M_1)$ e $L_2 = L(M_2)$
- Assumiamo che M_1 e M_2 siano TM con un solo nastro semi-infinito
- Costruiamo una TM a **due nastri nondeterministica** M :
 - Prova una suddivisione ^{non deterministica} dell' input $w = xy$
 - Muove y sul secondo nastro
 - Simula M_1 su x , M_2 su y
 - Accetta se entrambe accettano

M_{conc} inizia con l'input w sul Nastro 1. In uno stato non deterministico, M_{conc} sceglie una posizione all'interno di w e la considera il punto di divisione. La porzione prima di questa divisione è x . La porzione dopo questa divisione è y .

La porzione x rimane sul Nastro 1. La porzione y viene copiata e posizionata sul Nastro 2.

M_{conc} ora simula M_1 sul Nastro 1 (input x) e M_2 sul Nastro 2 (input y) in modo interleaved (alternato passo dopo passo).

M_{conc} accetta w solo se la simulazione di M_1 su x si ferma e accetta E la simulazione di M_2 su y si ferma e accetta.

Il non determinismo può essere usato nelle dimostrazioni per i Linguaggi Ricorsivi, purché si dimostri che la NTM che si costruisce termina sempre per ogni input, il che è possibile solo se tutti i rami di calcolo sono finiti.

Linguaggio RE: L'NTM può avere rami infiniti (loop), ma accetta se ne esiste uno finito accettante.
Linguaggio Ricorsivo: L'NTM deve garantire che tutti i suoi rami siano finiti (decisione garantita).

Poiché L1 e L2 sono Ricorsivi, le loro MT M1 e M2 sono algoritmi. Quando M_{conc} prende una coppia (x, y) e simula M1 su x e M2 su y, sa per certo che entrambe le simulazioni termineranno in un tempo finito, fornendo un risultato chiaro (Accetta o Rifiuta).

L'obiettivo è dimostrare che se L1 e L2 sono Ricorsivi, allora anche la loro concatenazione L1L2 è Ricorsiva (cioè, esiste una MT che la decide e che termina sempre).

Concatenazione / Ricorsivi

La nuova Macchina di Turing M_{conc} utilizza la stessa idea di fondo della MT non deterministica vista prima, ma la implementa in modo deterministico e completo.

- Si **prova** sistematicamente **ogni suddivisione** (sono finite) dell'input $w = xy$
- M_1 e M_2 **termineranno** per ogni x e y
- Si **accetta** se entrambe accettano per una qualche suddivisione
- **Rifiuta** se tutte le suddivisioni falliscono

Nota: non è sufficiente usare **nondeterminismo** per mostrare che **termino sempre!**

NTM RE: Una NTM riconosce un linguaggio se esiste un ramo accettante.
NTM Ricorsivo (Termina Sempre): Per decidere un linguaggio, la NTM deve garantire che tutti i rami di calcolo terminino (o accettando o rifiutando).
Poiché la NTM che si basa sulla divisione non deterministica può avere rami che vanno in loop infinito (nel caso RE), il solo non determinismo non è sufficiente a garantire la decidibilità per i ricorsivi.
Per i linguaggi Ricorsivi, l'uso della DTM con l'esplorazione sistematica e deterministica di tutte le opzioni, unita alla garanzia di terminazione delle MT sottostanti (M1 e M2), è ciò che assicura la decidibilità.

Il non determinismo permette a M^* di "coprire" tutte le possibili suddivisioni in parallelo. Se anche solo una di queste suddivisioni è valida, la NTM la trova e accetta. Questo garantisce la riconoscibilità del linguaggio L^* .

Stella di Kleene

- Le stesse idee continuano a funzionare
- **RE:** scommettere (usando il nondeterminismo) su una qualche suddivisione, accettare se M_1 accetta tutti i singoli pezzi
- **Ricorsivi:** sistematicamente si provano tutte le possibili suddivisioni (sono in numero finito) per cercarne uno in cui tutti i pezzi vengono riconosciuti

Una volta che un particolare ramo di calcolo ha scelto una divisione $w = w_1w_2...w_k$, M^* simula la MT originale M su ciascun pezzo w_i . M^* utilizza nastri o tracce ausiliarie per copiare w_1 , simulare M , copiare w_2 , simulare M , e così via, per tutti i k pezzi. M^* richiede che tutte e k le simulazioni di M si finiscano e accettino i rispettivi pezzi.

Reverse

- All'inizio si inverte l'input (come?)
- Poi si simula una TM per L sull'input invertito
- Funziona sia per **Ricorsivi** sia per **RE**

Decidibilità

- Macchine di Turing codificate come numeri
- Diagonalizzazione
- Problemi come linguaggi
- Problemi indecidibili

La Teoria della Computabilità utilizza l'Ordine Lessicografico per Lunghezza (o ordine standard):
- Si elencano tutte le stringhe per lunghezza crescente.
- Per le stringhe della stessa lunghezza, si usa l'ordine lessicografico (alfabetico).

Esempio ($\Sigma = \{0,1\}$):
Stringhe con Lunghezza 0: $\{\epsilon\}$; indice $j=1$
Stringhe con Lunghezza 1: $\{0,1\}$; indice $j=2,3$
Stringhe con Lunghezza 2: $\{00,01,10,11\}$; indice $j=4,5,6,7$
Stringhe con Lunghezza 3: $\{000,001,\dots\}$; indice $j=8,\dots$

L'ordine lessicografico puro (come in un dizionario) non funziona per enumerare, perché si rimarrebbe bloccati sulle stringhe di lunghezza 1.
Esempio (alfabeto $\{a, b\}$): Se usassimo l'ordine lessicografico puro, avremmo $a, aa, aaa, aaaa, \dots$ e non raggiungeremmo mai b o ab .

Enumerazione delle stringhe

L'enumerazione delle stringhe dipende solo dall'alfabeto Σ che si utilizza (tipicamente $\Sigma = \{0, 1\}$ o un alfabeto esteso di simboli).

- E' possibile definire un **ordinamento** di tutte le stringhe (finite) su un alfabeto:

- Per esempio, ordine lessicografico

Alfabeto (Σ): Un insieme finito di simboli.
Stringhe (Σ^*): L'insieme di tutte le stringhe di lunghezza finita che si possono formare usando i simboli di Σ (inclusa la stringa vuota ϵ). L'insieme Σ^* è infinito.

- Ci sono però infinite stringhe su un alfabeto

Per poter "contare" o enumerare queste stringhe, è necessario ordinarle

- Esiste quindi una **funzione biunivoca da numeri naturali a stringhe**:

- ha senso parlare di **j-esima stringa** (detta stringa di **indice j**)

Funzione Biunivoca: Poiché abbiamo definito un ordine ben preciso che permette di saltare da una stringa all'altra, possiamo associare ogni stringa ad un unico numero naturale (l'indice j).
Numerabilità: Questo dimostra che l'insieme di tutte le stringhe finite Σ^* è numerabile.

Ordine: Si applica l'ordine lessicografico per lunghezza (l'ordine standard) a tutte le stringhe binarie $\langle M \rangle$ che sono codifiche valide di Macchine di Turing.
Numerabilità: Poiché l'insieme di tutte le stringhe binarie è numerabile, anche il sottoinsieme di quelle stringhe che rappresentano MT valide è numerabile.

L'enumerazione delle Macchine di Turing dipende solo dal sistema di codifica scelto per rappresentare lo stato, le transizioni e gli alfabeti di una MT come una singola stringa $\langle M \rangle$

Univocità: Ogni Macchina di Turing M ha una singola e unica codifica $\langle M \rangle$.
Finitezza: La stringa $\langle M \rangle$ è di lunghezza finita.
Numerabilità: Poiché $\langle M \rangle$ è una stringa finita (su $\{0,1\}$), e l'insieme di tutte le stringhe finite è numerabile, anche l'insieme di tutte le Macchine di Turing è numerabile.
MT come Input: Questo è cruciale: una MT Universale U può prendere in input la stringa $\langle M \rangle$ e simulare il comportamento della macchina M che vi è codificata.

Enumerazione delle TM

- In modo simile è possibile definire un **ordinamento** di tutte le TM:
 - Per esempio, ordine lessicografico della loro descrizione (che è finita)
- Ci sono però infinite TM differenti (come sono infiniti i diversi FA e PDA)
- Esiste quindi **una funzione biunivoca da numeri naturali a TM**:
 - ha senso parlare di **i-esima TM** (detta TM di **indice i**)

Codifica: Ognuno di questi elementi della descrizione della TM può essere codificato in una singola stringa su un alfabeto binario $\{0,1\}$.

Finitezza della Descrizione: Una Macchina di Turing M è definita da un insieme finito di elementi: stati Q , alfabeto di input Σ , alfabeto del nastro Γ , stato iniziale q_0 , stati finali F , e la funzione di transizione δ .

Righe: Rappresentano l'enumerazione assoluta di tutte le Macchine di Turing (MT) possibili (codificate come stringhe $\langle M \rangle$).
Colonne: Rappresentano l'enumerazione assoluta di tutte le stringhe di input Σ^* (ordinate lessicograficamente per lunghezza).

Questa tabella organizza in modo sistematico l'insieme di tutte le possibili Macchine di Turing e l'insieme di tutti i possibili input (le stringhe).

Tabella di Accettazione

Stringa j $\longrightarrow$

1 2 3 4 5 6 . . .

TM i

- 1
- 2
- 3
- 4
- 5
- 6
-
-
-

X

Per la TM i , sulla riga i , vedo il linguaggio di quella TM (le stringhe che accetta/rifiuta)

Ogni cella (i, j) contiene un valore X che rappresenta il risultato della simulazione della MT i sull'input w_j .

- $X=1$ (Accetta): Significa che la i -esima Macchina di Turing si ferma e accetta la j -esima stringa (w_j) .
- $X=0$ (Non Accetta): Significa che la i -esima Macchina di Turing non accetta la j -esima stringa (w_j) .

Importante: Nella dimostrazione di Turing, "non accetta" copre due possibilità se la MT è solo un riconoscitore (Linguaggi RE):

1. La MT rifiuta w_j in tempo finito.
2. La MT entra in loop infinito su $\$w_j\$$.

$x=0$ significa che la
i-esima TM **non accetta**
la j-esima stringa;
 $x=1$ significa che la **accetta**

L'Uso della Diagonalizzazione
La tabella è utilizzata per costruire logicamente un nuovo linguaggio, L_DIAG, che è impossibile da riconoscere (o decidere) da qualsiasi MT presente nella lista.

- Si considera la Diagonale: Si prendono i valori X dove l'indice della riga i è uguale all'indice della colonna j.
- Si Inverte la Diagonale: Si definisce una nuova MT, M_DIAG, che ha la logica opposta a ciò che accade sulla diagonale.

> Se X=1 sulla diagonale (cioè MT_i accetta wj), M_DIAG NON accetta wj.
> Se X=0 sulla diagonale (cioè MT_i non accetta wj), M_DIAG accetta wj.

- Il Paradosso: Se L_DIAG fosse riconoscibile (L RE) o decidibile (L Ricorsivo), la sua MT, M_DIAG, dovrebbe esistere da qualche parte nella lista infinita: diciamo che sia MT_k.

> Cosa fa MT_k (cioè M_DIAG) quando riceve il suo input wk?
>> Se MT_k accetta wk, allora la sua definizione (inversione della diagonale) dice che non dovrebbe accettare.
>> Se MT_k non accetta wk, allora la sua definizione dice che dovrebbe accettare.

Poiché M_DIAG differisce da tutte le MT della lista in almeno un punto (quello sulla diagonale), M_DIAG non può esistere nella lista. Questo dimostra l'esistenza di un linguaggio (e quindi di un problema) che non può essere riconosciuto da nessuna Macchina di Turing, stabilendo l'indcidibilità come un concetto fondamentale.

TM

Diagonalizzazione

Stringhe

	1	2	3	4	5	...
1	1	0	0	1	0	...
2		1				
3			0			
4				0		
5					1	
...						...

Diagonalizzazione

Inverti i
valori nella
diagonale

TM

Stringhe

	1	2	3	4	5	...
1	0	0	0	1	0	...
2		0				
3			1			
4				1		
5					0	
...						...

Diagonalizzazione

Inverti i
valori nella
diagonale

TM

Stringhe

	1	2	3	4	5	...
1	0	0	0	1	0	...
2		0				
3			1			
4				1		
5					0	
...						...
	0	0	1	1	0	...

Non lo si può
trovare in una
riga! –
è **diversa da**
tutte le righe
almeno per
un valore

Linguaggio che non può
essere riconosciuto da
nessuna Macchina di Turing

?

Dettagli del ragionamento per Diagonalizzazione

- Ogni volta che abbiamo una tabella come la precedente, possiamo **diagonalizzarla**
 - Cioè, costruire una sequenza D complementando ogni bit lungo la diagonale principale
- Formalmente, $D = a_1 a_2 \dots$, con:
 - $a_i = 0$ se la casella (i,i) contiene 1,
 - $a_i = 1$ altrimenti

La macchina MT_j che riconosce per assurdo L_D (la j -esima riga), quando elabora la stringa w_j (la j -esima colonna), produce un risultato che è sempre l'inverso del risultato che dovrebbe produrre la macchina che riconosce L_D . In altre parole: Valore della cella (j, j) not equal Valore atteso da L_D nella cella (j, j) . Poiché MT_j (o qualsiasi altra MT MT_j) non può riconoscere un linguaggio da cui differisce su un input cruciale, e L_D è costruito per differire da tutte le MT (sulla diagonale), nessuna MT può riconoscere L_D .

L'obiettivo è dimostrare che esiste un linguaggio, chiamato L_D (il linguaggio diagonale), che non può essere riconosciuto da nessuna Macchina di Turing, e quindi non è Ricorsivamente Enumerabile (non è RE).

La Dimostrazione per Diagonalizzazione

- Può D essere una riga della tabella?
 - cioè essere il linguaggio accettato da una TM?
- Supponiamo che sia la j -esima riga
 - ma D contiene nella j -esima colonna un valore diverso rispetto a quello della j -esima riga!
- Questo vale per ogni j
- Quindi D non è una riga (rappresenta un linguaggio che **nessuna TM riconosce**)

La diagonale (i,i) considera il caso in cui la macchina di Turing (MT_i) ha come input la sua stessa codifica come stringa (w_i).

Linguaggio di Diagonalizzazione

L_d è il linguaggio composto da tutte quelle Macchine di Turing che non riescono ad accettare sé stesse.

- Linguaggio di diagonalizzazione è quindi
$$L_d = \{w \mid w \text{ è la stringa } i\text{-esima,} \\ \text{e la } i\text{-esima TM } \mathbf{non} \text{ riconosce } w\}$$
- Abbiamo mostrato che L_d **non è** un linguaggio **ricorsivamente enumerabile**
- cioè non esiste una TM M tale che $L(M) = L_d$

L_d è un linguaggio (un insieme di stringhe) che raccoglie solo le codifiche di quelle Macchine di Turing che, se eseguite sulla loro propria descrizione come input, non si fermano nello stato di accettazione.

Graficamente

Linguaggi non
ricorsivamente
enumerabili

Linguaggi
ricorsivamente
enumerabili

Problemi
decidibili =
Linguaggi
ricorsivi

L_d

Ci sono linguaggi
RE, ma non
ricorsivi?

Dalla teoria alla pratica

- Il fatto che L_d sia indecidibile è interessante dal punto di vista **teorico**, ma non ha impatto sul mondo reale
- Considereremo altri problemi legati alle TM, per mostrare che alcuni **problemi reali sono indecidibili**

Esempio: Problemi Indecidibili

- Può una certa linea di codice essere eseguita durante l'esecuzione di un programma?
- Può una variabile contenere un certo valore durante l'esecuzione di un programma?

Il Linguaggio Universale

- Un esempio di un linguaggio ricorsivamente enumerabile, ma non ricorsivo, è il linguaggio L_u della **Macchina di Turing Universale**

La UTM è una singola Macchina di Turing che può simulare il comportamento di qualsiasi altra MT.

- La **UTM** ha come input l'**indice i** di una certa TM M e l'**indice j** di una stringa w , e accetta se e solo se **M_i accetta w_j**
- UTM è una semplice **simulazione** (realizzabile con più nastri)

L_u : linguaggio universale (definizione formale)

è la stringa codificata che rappresenta la coppia formata dalla descrizione di una MT e da un input per quella MT.

- Supponiamo di **enumerare le coppie** $\langle i, j \rangle$ e le rappresentiamo con corrispondenti stringhe $\mathbf{w}_{\langle i, j \rangle}$

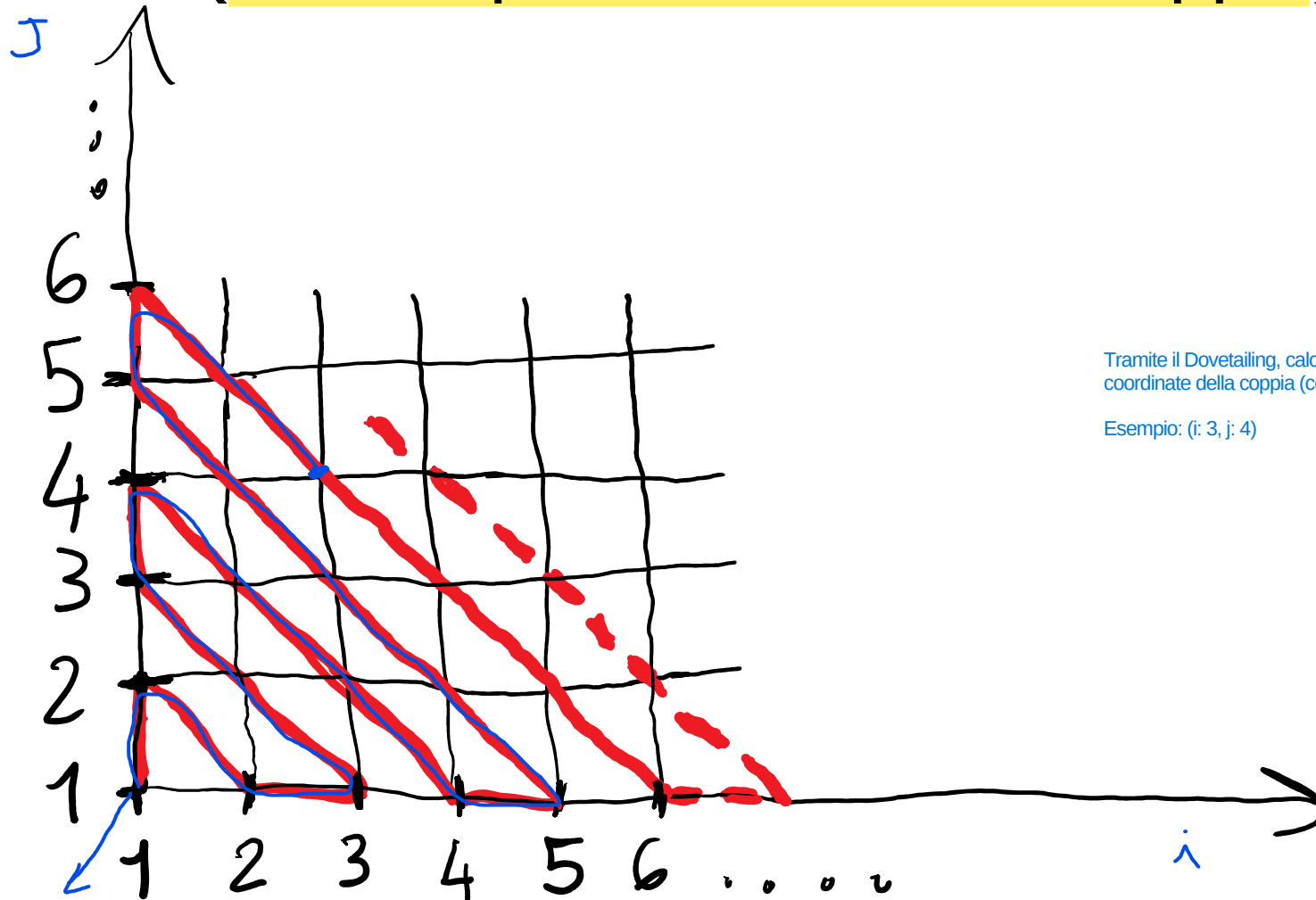
Dai due indici i e j , ottengo un indice $\langle i, j \rangle$ tramite Dovetailing e dall'indice composto, trovo la stringa associata

- $L_u = \{ \mathbf{w}_{\langle i, j \rangle} \mid M_i \text{ accetta } w_j \}$
- UTM garantisce che L_u è sicuramente RE
- Mostreremo che L_u **non** può essere **ricorsivo**

La tecnica del Dovetailing (incastro a coda di rondine) fornisce proprio il metodo per definire un indice composto univoco (un singolo numero naturale k) partendo da una coppia di indici naturali (i, j) .

Dovetailing

(tecnica per enumerare le coppie)



Tramite il Dovetailing, calcolo la lunghezza da origine a coordinate della coppia (così da andare da $N \times N$ a N)

Esempio: $(i: 3, j: 4)$

Calcolo la lunghezza della traccia blu disegnata sul grafico per ottenere il nuovo indice composto

Prova

Questo significa che esiste una Macchina di Turing Decisore che termina sempre (in accettazione o rifiuto) e riconosce L_u .

Ipotizziamo per assurdo che L_u sia ricorsivo

Stiamo costruendo un ipotetico decisore D_d per L_d che usa l'ipotetico decisore D_u per L_u .

Data una **stringa w** , possiamo decidere se è in L_d nel seguente modo

Calcoliamo a quale indice corrisponde w :

cioè consideriamo **j tale che $w = w_j$**

la stringa di input w è una codifica di una Macchina di Turing, e viene testata su sé stessa

Verifichiamo se **$w_{\langle j, j \rangle}$ non appartiene a L_u**

(possibile per ipotesi) e in tal caso concludiamo che

w è in L_d

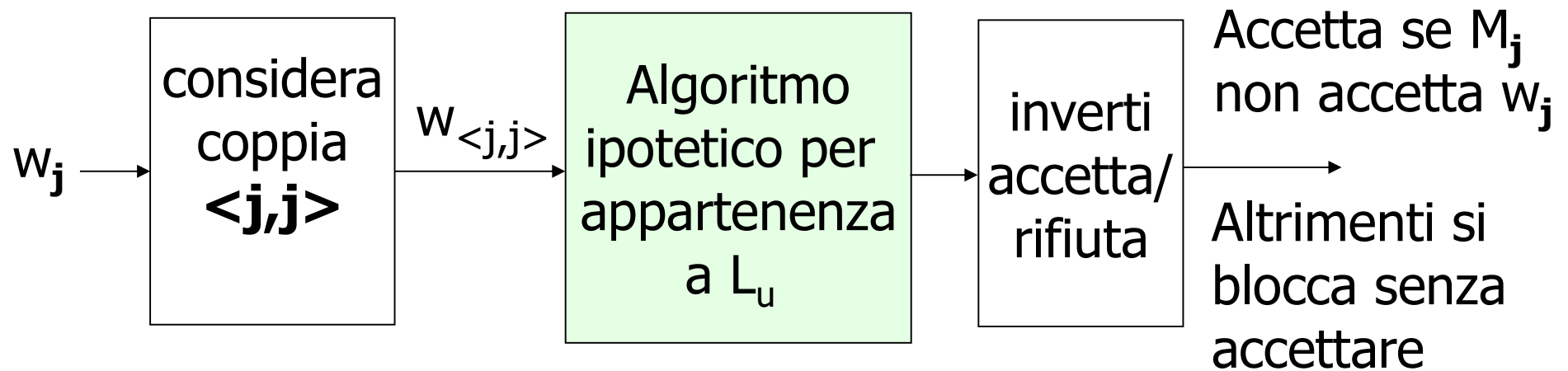
Per definizione di L_d , la stringa w appartiene a L_d (w è la descrizione di una macchina che non accetta sé stessa).

Ma questo non è possibile!

Conclusione: L_u è RE, ma non è ricorsivo

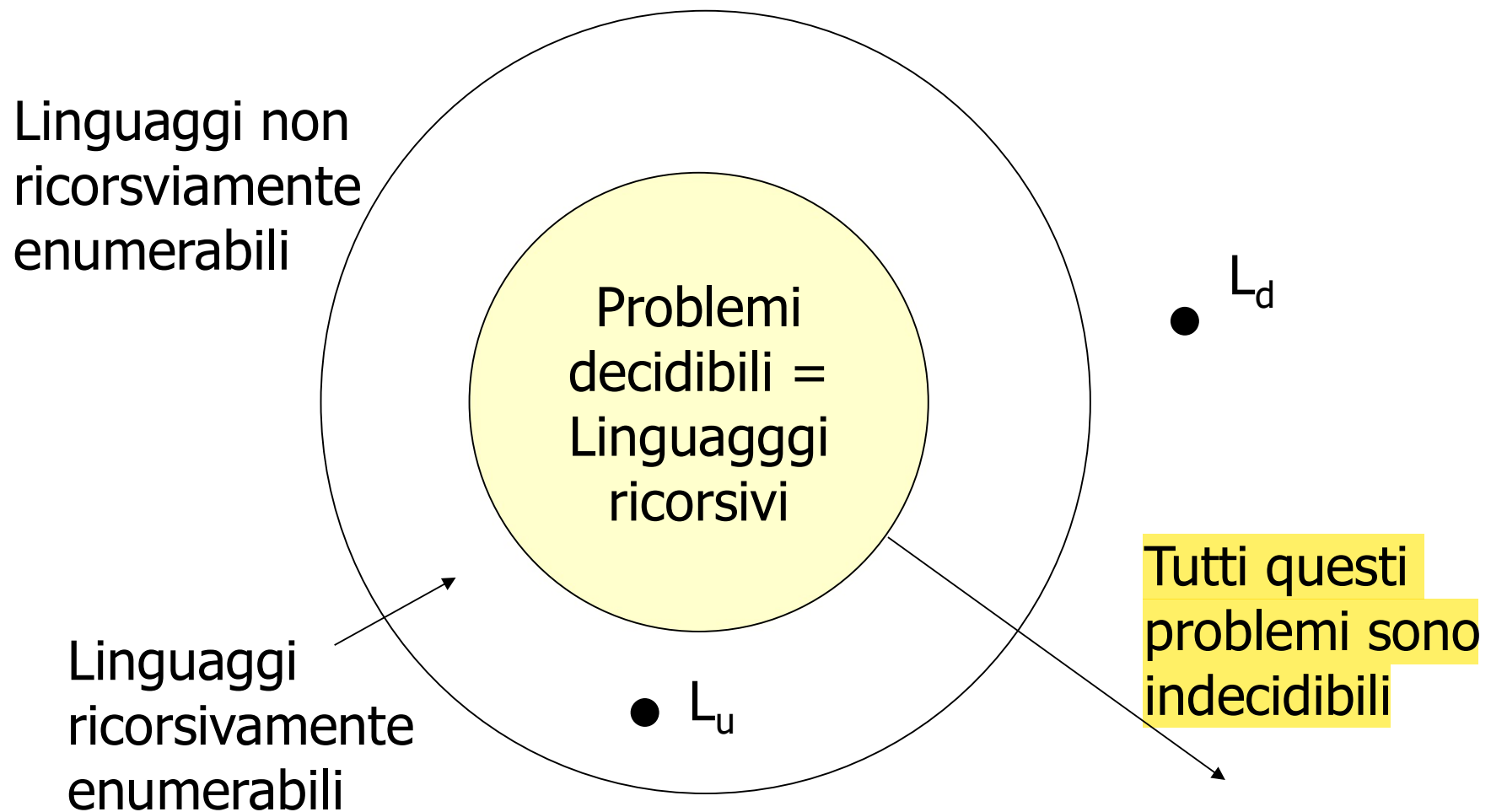
La costruzione della macchina decisore D_d è fatta apposta per usare il decisore di L_u come un test di accettazione di sé, e poi invertire il risultato per soddisfare la definizione di L_d . È l'atto di inversione (passare da "non appartiene a L_u " ad "appartiene a L_d ") che genera la contraddizione: se la macchina che decide L_u esistesse, potremmo usarla per decidere L_d , ma ciò è impossibile.

Immagine della Riduzione da L_d a L_u



Questo sarebbe un algoritmo per L_d , che non può esistere!

Graficamente



Equivalenza: Un linguaggio di programmazione Turing-completo (come C, Java, Python, ecc.) può essere visto come una codifica di una Macchina di Turing M. La chiamata alla funzione con i parametri è l'input w.
Indecidibilità: Non esiste un algoritmo generale (un debugger o analizzatore statico perfetto) che possa prendere in input qualsiasi programma e qualsiasi input e garantire di dirti se quel programma si fermerà o entrerà in un loop infinito.
Conseguenza: È impossibile scrivere un compilatore che avvisi sempre se il tuo programma finirà in loop.

Se un problema equivale logicamente a determinare se una Macchina di Turing accetta un input (cioè se è in Lu), allora quel problema è indecidibile.

Problemi reali

Problema dell'Arresto o Halting Problem

Indecidibilità di L_u (problema della fermata):

es. è indecidibile il problema di stabilire se, dato il codice di una funzione (es in C) ed il valore dei parametri, l'esecuzione di tale codice terminerà

è indecidibile anche il problema di stabilire se, dato un programma, il nome x di una variabile e un valore v, x può valere v durante l'esecuzione

es. nel main uso una variabile booleana **x** inizialmente **false**, chiamo la funzione sopra ed assegno, al ritorno da tale funzione, **x=true**

La Proprietà: "La variabile x raggiunge il valore v." Questa è una proprietà della semantica (comportamento dinamico) del programma.
Non Banale: Non è una proprietà soddisfatta da tutti i programmi né da nessun programma.
Indecidibilità: Non esiste un algoritmo che possa risolvere questo problema per tutti i programmi possibili.

Indecidibilità (Teorema di Church): Il teorema di Church (1936) dimostrò che non esiste un algoritmo generale che, data una formula arbitraria della logica del primo ordine, possa decidere se essa sia valida o meno.

Problemi reali

■ È indecidibile il problema di stabilire se, data una **formula della logica dei predicati** essa è, o meno, **valida**

- Programmazione logica (es. PROLOG) è **Turing** equivalente (può codificare qualsiasi procedura)

Può simulare qualsiasi Macchina di Turing, e quindi è soggetto ai limiti di indecidibilità.

- un **programma logico** è una formula della logica dei predicati

■ la **terminazione** con successo dell'esecuzione, per un **goal**, si ha quando formula totale è **valida**

È la formula che si vuole dimostrare.

■ riduco problema fermata a problema validità

Se esistesse un algoritmo D_Validità per decidere la validità di qualsiasi formula phi, potremmo usarlo per risolvere il Problema dell'Arresto. Poiché siamo in grado di decidere l'Arresto, e sappiamo che questo è impossibile, ne consegue che l'algoritmo D_Validità non può esistere.

E' possibile liberarsi delle computazioni non terminanti?

- Consideriamo un ipotetico **linguaggio di programmazione** con il quale è possibile scrivere **solli algoritmi**
 - cioè un linguaggio di programmazione in cui un programma è **garantito terminare sempre (accettando o rifiutando)**
 - è possibile che esista un tale linguaggio di programmazione e che consenta di esprimere **tutti i linguaggi ricorsivi?**
 - cioè tutti i problemi risolubili alitmicamente?

Diagonalizzazione

tutti i programmi
di un ipotetico
linguaggio di
programmazione
che produce soli
algoritmi

Stringhe

	1	2	3	4	5	...
1	1	0	0	1	0	...
2		1				
3			0			
4				0		
5					1	
...						...

Diagonalizzazione

Inverti i
valori nella
diagonale

tutti i programmi
di un ipotetico
linguaggio di
programmazione
che produce soli
algoritmi

Stringhe

	1	2	3	4	5	...
1	0	0	0	1	0	...
2		0				
3			1			
4				1		
5					0	
...						...

Questo dimostra che, se si vuole creare un linguaggio di programmazione che abbia la potenza di esprimere tutti i problemi decidibili (cioè, che sia Turing-equivalente nel senso dei Decisori), quel linguaggio deve includere la possibilità di computazioni non terminanti (loop). Non c'è modo di sbarazzarsi dei loop infiniti senza perdere la capacità di esprimere alcuni problemi che sono, di per sé, completamente risolvibili (ad esempio, un linguaggio di programmazione senza while e con solo for non è Turing equivalente).

Diagonalizzazione

Inverti i
valori nella
diagonale

tutti i programmi
di un ipotetico
linguaggio di
programmazione
che produce soli
algoritmi

Stringhe →

	1	2	3	4	5	...
1	0	0	0	1	0	...
2		0				
3			1			
4				1		
5					0	
...						...

Assurdo! E' un
linguaggio
ricorsivo non
esprimibile nel
linguaggio di
programmazione

(data w_j basta
darla in input
all'algoritmo j e
invertire la
risposta)

L'ipotesi che L_DEC possa esprimere tutti i linguaggi ricorsivi porta a una contraddizione: Esiste almeno un linguaggio Ricorsivo ($L_DEC-DIAG$) che non può essere espresso da L_DEC .

Altri Problemi Indecidibili

Proprietà di linguaggi RE (riconosciuti da TM)

- Decidibilità di problemi su linguaggi RE (e quindi riconosciuti da una TM M):
 - $E' L(M)$ un linguaggio regolare?
 - $E' L(M)$ un CFL?
 - $L(M)$ include le stringhe palindrome?
 - $E' L(M)$ vuoto?
 - $L(M)$ contiene più di 1000 stringhe?
 - $L(M) = L(M')$?
 - Ecc., ecc.

Proprietà P: Una proprietà P è un sottoinsieme della classe di tutti i linguaggi RE, ovvero P sottoinsieme di tutti i linguaggi RE.

Proprietà di linguaggi RE (riconosciuti da TM)

- È decidibile se, data una MDT M, $L(M)$ soddisfa o meno una proprietà **P**?
 - esempio: P = essere vuoto
- Ciò è come chiedersi:
 - considerato $L_P = \{ w_i \mid L(M_i) \text{ soddisfa } P \}$,
 - L_P è **ricorsivo** oppure no?

Dato in input un indice i, voglio capire se il Linguaggio della TM_i rispetta proprietà P



Il linguaggio L_P viene definito come il problema di decidere se una Macchina di Turing (MT) ha quella proprietà

Teorema di Rice

Il Teorema di Rice riguarda la decidibilità (il fatto di essere Ricorsivo) di questo linguaggio L_P .

Una proprietà P è non banale se non è soddisfatta da tutti i linguaggi RE, né è soddisfatta da nessun linguaggio RE. In termini più semplici, la classe P non può essere l'insieme vuoto o l'insieme di tutti i linguaggi RE. Se P è non banale, allora il linguaggio L_P (il problema di decidere se $L(M)$ ha la proprietà P) non è Ricorsivo (è indecidibile).

- Ci sono due casi di proprietà P **banali** per cui L_P è ricorsivo

- **proprietà P mai soddisfatta** ($L_P = \emptyset$) Nessun linguaggio RE soddisfa P . L_P è Ricorsivo? Sì, perché $L_P =$ insieme vuoto, ed è sempre decidibile.
- **proprietà P sempre soddisfatta** ($L_P =$ tutte le stringhe) Ogni linguaggio RE soddisfa P . L_P è Ricorsivo? Sì, perché $L_P = \Sigma^*$ (l'insieme di tutte le stringhe, ovvero tutte le codifiche $\langle M \rangle$), ed è sempre decidibile.

- **Teorema di Rice:** per tutte le altre proprietà P (non banali) L_P è non ricorsivo

Non esiste un algoritmo generale che possa determinare se un qualsiasi programma possiede una di queste proprietà.

Applicazioni del Teorema di Rice

- Tutti i problemi interessanti su linguaggi RE (e quindi di TM) sono **indecidibili**:
 - E' $L(M)$ un linguaggio regolare?
 - E' $L(M)$ un CFL?
 - $L(M)$ include le stringhe palindrome?
 - E' $L(M)$ vuoto?
 - $L(M)$ contiene più di 1000 stringhe?
 - $L(M) = L(M')$?
 - Ecc., ecc.

Applicazioni del Teorema di Rice

- Solo **proprietà banali** sono **decidibili**:

- E' $L(M)$ un linguaggio RE? Per definizione, la Macchina di Turing M riconosce il suo linguaggio $L(M)$. Il linguaggio riconosciuto da una MT è, per definizione, un linguaggio RE.
 - Dato M è **sempre vero**
(quindi algoritmo banale che risponde sempre sì)
- E' $L(M)$ non RE? Ancora una volta, per definizione, il linguaggio $L(M)$ è riconosciuto da una MT, quindi non può essere un linguaggio non RE.
 - Dato M è **sempre falso**
(quindi algoritmo banale che risponde sempre no)

- Proprietà Semantiche (Riguardano $L(M)$): Queste sono le proprietà a cui il Teorema di Rice SI APPLICA. Riguardano ciò che fa la macchina in termini di input e output finale, indipendentemente da come ci arriva.
Definizione: Se due Macchine di Turing, $M1$ e $M2$, riconoscono lo stesso linguaggio ($L(M1) = L(M2)$), allora devono avere la stessa proprietà semantica P .
Risultato del Teorema di Rice: Tutte queste proprietà (se non banali) sono indecidibili.
- Proprietà Strutturali o Dinamiche (Non Riguardano Solo $L(M)$): Queste sono le proprietà a cui il Teorema di Rice NON SI APPLICA. Riguardano la macchina M come codice o come processo di calcolo, non solo il suo risultato finale.
Definizione: È possibile che due Macchine di Turing $M1$ e $M2$ che riconoscono lo stesso linguaggio ($L(M1) = L(M2)$) abbiano proprietà diverse in questa categoria.

Esempi a cui il Teorema di Rice non si applica

■ Problemi che riguardano caratteristiche di una TM **non esclusivamente riguardanti il linguaggio che riconosce:**

- M riconosce w in 10 passi? Questa è una proprietà della macchina M in quel momento, non del linguaggio $L(M)$. Due macchine diverse $M1$ e $M2$ potrebbero riconoscere lo stesso linguaggio $L(M1) = L(M2)$, ma una potrebbe accettare w in 5 passi e l'altra in 15.
- M è la i -esima TM (cioè $M = M_i$)? Questa è una proprietà puramente sintattica sulla codifica $\langle M \rangle$.
- M_i riconosce w_i ? Sebbene sia una proprietà relativa all'accettazione, non è una proprietà generale del linguaggio $L(M_i)$ inteso come insieme, ma è una proprietà specifica della coppia $\langle M_i, w_i \rangle$.

Si chiede se una stringa w appartiene a un insieme L

Da decidibilità problemi su linguaggi a calcolabilità funzioni

Si chiede se una funzione f che rappresenta il problema può essere calcolata da una Macchina di Turing.

- In altre trattazioni,

la decidibilità di un problema (linguaggio) e la calcolabilità di una funzione sono due modi di esprimere lo stesso concetto di risolubilità algoritmica.

LA **decidibilità del problema** se, dato un i , $w_i \in L$ espressa come:

calcolabilità della funzione $f: \mathbb{N} \rightarrow \{0,1\}$
in cui $f(i)=1$ corrisponde al fatto che $w_i \in L$

- **f** si dice **calcolabile** quando il problema è **decidibile** (calcolato da TM che termina sempre)
- **f** si dice **parzialmente calcolabile** quando il problema è **semi-decidibile** (calcolato da TM)

Da decidibilità problemi su linguaggi a calcolabilità funzioni

- Concetto calcolabilità di funzioni **esteso a:**
calcolabilità di funzioni $f: \mathbb{N} \rightarrow \mathbb{N}$
Quando la TM termina, il nastro contiene la rappresentazione del numero naturale $f(i)$.
- **f** si dice **calcolabile** quando esiste una TM che termina sempre e che, su input i , dà **risultato $f(i)$**
- **invece** che TM che **accettano o meno**: TM che danno un risultato (sul nastro quando terminano)
- Quando si ha a che fare con una funzione o un predicato (es. in modello logica predicati) bisogna sempre **chiedersi** se sia **calcolabile**

Importanza: Tutte le domande sull'esistenza di un algoritmo per risolvere un problema (logico, matematico o informatico) si riducono a questa singola domanda: La funzione che descrive la soluzione è calcolabile?
Indecidibilità: Se la funzione è incalcolabile (come la funzione caratteristica del Problema dell'Arresto), allora non esiste alcun algoritmo (Macchina di Turing) che possa risolverla.

Esercizi

- Si consideri il seguente linguaggio
$$L = \{ w_{\langle x,y \rangle} \mid x \text{ e } y \text{ sono tali che } L(M_x) \text{ e } L(M_y) \text{ hanno almeno una stringa in comune} \}$$
dove M_x e M_y sono la x -esima e la y -esima macchina di Turing.

Dire se L è ricorsivo, ricorsivamente enumerabile, o nemmeno ricorsivamente enumerabile. Giustificare la risposta.

Esercizi

- Si consideri il seguente linguaggio
$$L = \{ w_{\langle x,y \rangle} \mid x \text{ e } y \text{ sono tali che } L(M_x) \text{ e } L(M_y) \text{ **non** hanno stringhe in comune} \}$$
dove M_x e M_y sono la x -esima e la y -esima macchina di Turing.

Dire se L è ricorsivo, ricorsivamente enumerabile, o nemmeno ricorsivamente enumerabile. Giustificare la risposta.